BIDS Series Classification: ML Analysis Report

This notebook is a **presentation-quality analysis** of the automated BIDS classification system. It covers:

1. The clinical problem motivating automated series labeling
2. How the heuristic inference pipeline works and what label classes it produces
3. How the XGBoost model was trained and how well it performs overall
4. Per-class and per-family performance, hard classes, and unknown-label coverage

> All data loaded here is pre-computed by `ml.ipynb` and stored in `outputs/`. Re-run `ml.ipynb` then the **"Analysis Data Export"** cell before refreshing results here.

In [1]:
```
from pathlib import Path
import warnings

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
import matplotlib.patches as mpatches

warnings.filterwarnings("ignore", category=FutureWarning)
pd.set_option("display.max_columns", 40)
pd.set_option("display.float_format", "{:.4f}".format)

plt.rcParams.update({
    "figure.dpi": 130,
    "axes.spines.top": False,
    "axes.spines.right": False,
    "font.size": 11,
    "axes.titlesize": 13,
    "axes.labelsize": 11,
})

# Locate project root (the directory containing src/)
_cwd = Path(".").resolve()
PROJECT_DIR = _cwd
while not (PROJECT_DIR / "src").exists() and PROJECT_DIR.parent != PROJECT_D
    PROJECT_DIR = PROJECT_DIR.parent

ANALYSIS_DATA_DIR = PROJECT_DIR / "outputs" / "analysis_data"
FINAL_METRIC_DIR  = PROJECT_DIR / "outputs" / "plots" / "finalized_model" /
CONFUSION_DIR     = PROJECT_DIR / "outputs" / "plots" / "finalized_model" /
REVIEW_DIR        = PROJECT_DIR / "outputs" / "review"

_family_palette = {
```

```
    "anat": "#2B6CB0", "func": "#48BB78", "dwi": "#9F7AEA",
    "perf": "#ED8936", "fmap": "#38B2AC", "localizer": "#E53E3E",
    "exclude": "#718096", "unknown": "#CBD5E0",
}

print(f"Project root : {PROJECT_DIR}")
print(f"Analysis data: {ANALYSIS_DATA_DIR}")
```

```
Project root : /Users/cmokashi/Documents/GitHub/find_BIDS
Analysis data: /Users/cmokashi/Documents/GitHub/find_BIDS/outputs/analysis_d
ata
```

In [2]:

```
# Load all pre-computed artifacts
analysis_df    = pd.read_csv(ANALYSIS_DATA_DIR / "heuristic_scores_with_spli
split_summary  = pd.read_csv(ANALYSIS_DATA_DIR / "split_summary.csv")
final_metrics  = pd.read_csv(FINAL_METRIC_DIR  / "final_model_eval_metrics.c
report_test    = pd.read_csv(FINAL_METRIC_DIR  / "classification_report_infe
report_val     = pd.read_csv(FINAL_METRIC_DIR  / "classification_report_infe
unknown_review = pd.read_csv(REVIEW_DIR        / "unknown_joint_label_infere

# Derive the ML-style joint label from heuristic columns
analysis_df["joint_label"] = (
    analysis_df["inferred_datatype"].fillna("unknown").astype(str)
    + "::"
    + analysis_df["inferred_suffix"].fillna("unknown").astype(str)
)
analysis_df["is_heuristic_unknown"] = (
    analysis_df["inferred_datatype"].fillna("").str.lower().str.contains("ur
    | analysis_df["inferred_suffix"].fillna("").str.lower().str.contains("ur
)

# Use heuristic-unknown flag to separate known vs unknown series
labeled_df  = analysis_df[~analysis_df["is_heuristic_unknown"]].copy()
excluded_df = analysis_df[analysis_df["is_heuristic_unknown"]].copy()

_summary_labels = {"accuracy", "macro avg", "weighted avg"}
class_report = report_test[~report_test["label"].isin(_summary_labels)].copy
class_report["datatype"] = class_report["label"].str.split("::").str[0]
class_report["support"]  = class_report["support"].astype(int)

print(f"Total series rows          : {len(analysis_df):,}")
print(f"  Heuristic-labeled (known): {len(labeled_df):,}")
print(f"  Heuristic-unknown        : {len(excluded_df):,}")
display(split_summary)
```

```
Total series rows          : 144,358
  Heuristic-labeled (known): 120,998
  Heuristic-unknown        : 23,360
```

	split	rows
0	all	77489
1	test	11622
2	train	45474
3	val	9773

Part 1 — Problem Statement

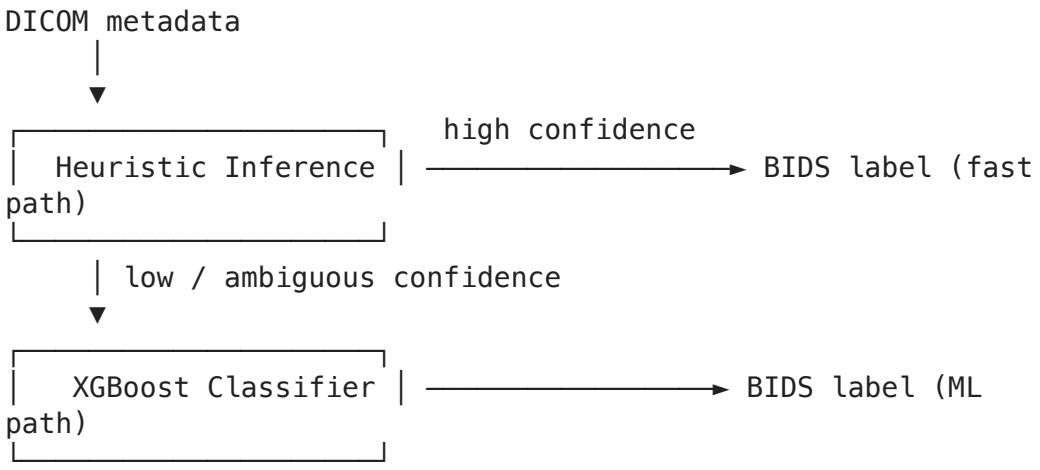
The Challenge of BIDS Organization

The Brain Imaging Data Structure (BIDS) standard organizes neuroimaging acquisitions by assigning every acquired series a **datatype** (broad scan family: `anat`, `func`, `dwi`, …) and a **suffix** (specific contrast or modality: `T1w`, `bold`, `dwi`, …).

Manual BIDS labeling is:

- **Slow** — a researcher must inspect series descriptions, acquisition parameters, and DICOM metadata for every series
- **Error-prone** — naming conventions vary widely across scanners, sites, and research groups
- **Not scalable** — large multi-site studies routinely contain thousands of series

Two-Stage Solution



1. **Heuristic inference** — a rule-based engine scores each series against BIDS schema rules, producing a provisional `inferred_datatype` and `inferred_suffix` with an associated `min_confidence` score.

2. **ML correction** — an XGBoost classifier trained on high-confidence heuristic labels helps classify ambiguous series. It predicts the **joint label** `inferred_datatype::inferred_suffix` (e.g., `anat::T1w`, `dwi::dwi`, `fmap::fieldmap`).

Part 2 — The Heuristic Inference Pipeline

Label Space

The system classifies every series into one of the following `datatype::suffix` combinations:

Datatype	Description	Key Suffixes
`anat`	Structural / anatomical scans	T1w, T2w, FLAIR, T2starw, PDw, angio
`func`	Functional MRI	bold, sbref, funcStatMap, funcTimeSeries
`dwi`	Diffusion-weighted imaging	dwi, sbref
`perf`	Perfusion / ASL imaging	m0scan, perfCbfLikeMap, perfCbvLikeMap, perfTimingMap
`fmap`	Field maps	fieldmap, magnitude, phasediff, epi, fmapSusceptibilityOrPhaseMap
`localizer`	Scout / planning scans	localizer
`unknown`	Series the heuristic engine cannot classify	unknown

Confidence Score

Each heuristic assignment carries a `min_confidence` score $\in [0, 1]$ — the minimum of the datatype and suffix confidence scores. Rows where `min_confidence` is low *or* where either label part is `unknown` are the primary target of the ML correction step.

In [3]:
```
# Class catalog: unique joint labels in the ML-labeled portion of the datase
train_classes = sorted(labeled_df["joint_label"].dropna().unique().tolist())
catalog_rows = []
for jl in train_classes:
    parts = jl.split("::")
    catalog_rows.append({"datatype": parts[0], "suffix": parts[1] if len(par
catalog_df = pd.DataFrame(catalog_rows)

family_catalog = (
    catalog_df.groupby("datatype")["suffix"]
    .apply(list)
```

```
    .reset_index()
    .rename(columns={"suffix": "suffixes"})
)
family_catalog["n_suffixes"] = family_catalog["suffixes"].apply(len)
family_catalog["suffixes"]   = family_catalog["suffixes"].apply(lambda s: ",

print(f"Total joint-label classes in labeled data: {len(train_classes)}")
display(family_catalog)
```

```
Total joint-label classes in labeled data: 28
```

	datatype	suffixes	n_suffixes
0	anat	FLAIR, PD, SWI, T1w, T2starw, T2w, anatReforma...	8
1	dwi	dwi, dwiOtherDerived, dwiParamMap, dwiSegmenta...	4
2	fmap	epi, fmapOtherDerived, fmapReformat, fmapSegme...	7
3	func	bold, funcStatMap, sbref	3
4	localizer	localizer	1
5	perf	dce, dsc, perfCbfLikeMap, perfCbvLikeMap, perf...	5

Part 2B — Heuristic vs ML Shift Analysis

This section quantifies how ML changes predictions relative to the heuristic engine on the comparable population (`ml_split == "all"` with both labels and confidences present).

We evaluate:

- Continuous confidence shift: `ml_pred_confidence − min_confidence`
- Full joint-label changes: `heuristic_joint_label != ml_pred_joint_label`
- Data-driven confidence blocks from quantile bins

In [28]:
```
# Build comparable heuristic vs ML frame (deduplicated by using ml_split ==
comp_df = analysis_df[
    (analysis_df["ml_split"] == "all")
    & analysis_df["min_confidence"].notna()
    & analysis_df["ml_pred_confidence"].notna()
    & analysis_df["joint_label"].notna()
    & analysis_df["ml_pred_joint_label"].notna()
].copy()

def _normalize_joint_label(lbl):
    if pd.isna(lbl):
        return np.nan
    s = str(lbl).strip()
    if "::" in s:
        return s
    if "_" in s:
```

```
        left, right = s.split("_", 1)
        return f"{left}::{right}"
    return s

comp_df["heuristic_joint_label"] = comp_df["joint_label"].map(_normalize_joi
comp_df["ml_joint_label"] = comp_df["ml_pred_joint_label"].map(_normalize_jo
comp_df["confidence_delta"] = comp_df["ml_pred_confidence"] - comp_df["min_c
comp_df["abs_delta"] = comp_df["confidence_delta"].abs()
comp_df["label_changed"] = comp_df["heuristic_joint_label"] != comp_df["ml_j
comp_df["delta_direction"] = np.select(
    [
        comp_df["confidence_delta"] > 0,
        comp_df["confidence_delta"] < 0,
    ],
    ["boosted", "reduced"],
    default="unchanged",
)

n_total = len(comp_df)
n_changed = int(comp_df["label_changed"].sum())
n_boosted = int((comp_df["confidence_delta"] > 0).sum())
n_reduced = int((comp_df["confidence_delta"] < 0).sum())
n_tied = int((comp_df["confidence_delta"] == 0).sum())

headline = pd.DataFrame([
    {
        "comparable_rows": n_total,
        "changed_labels": n_changed,
        "changed_rate": n_changed / n_total if n_total else np.nan,
        "boosted_confidence": n_boosted,
        "boosted_rate": n_boosted / n_total if n_total else np.nan,
        "reduced_confidence": n_reduced,
        "reduced_rate": n_reduced / n_total if n_total else np.nan,
        "unchanged_confidence": n_tied,
        "unchanged_conf_rate": n_tied / n_total if n_total else np.nan,
        "delta_mean": float(comp_df["confidence_delta"].mean()) if n_total e
        "delta_median": float(comp_df["confidence_delta"].median()) if n_tot
    }
])

print("Comparable population built (ml_split == 'all').")
display(headline.style.format({
    "changed_rate": "{:.2%}",
    "boosted_rate": "{:.2%}",
    "reduced_rate": "{:.2%}",
    "unchanged_conf_rate": "{:.2%}",
    "delta_mean": "{:.4f}",
    "delta_median": "{:.4f}",
}))
```

```
Comparable population built (ml_split == 'all').
```

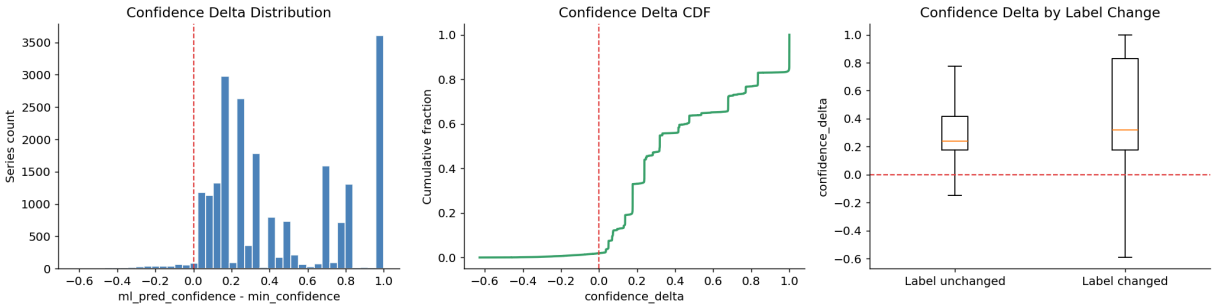
	comparable_rows	changed_labels	changed_rate	boosted_confidence	boosted_rate
0	21395	18722	87.51%	20980	98.06%

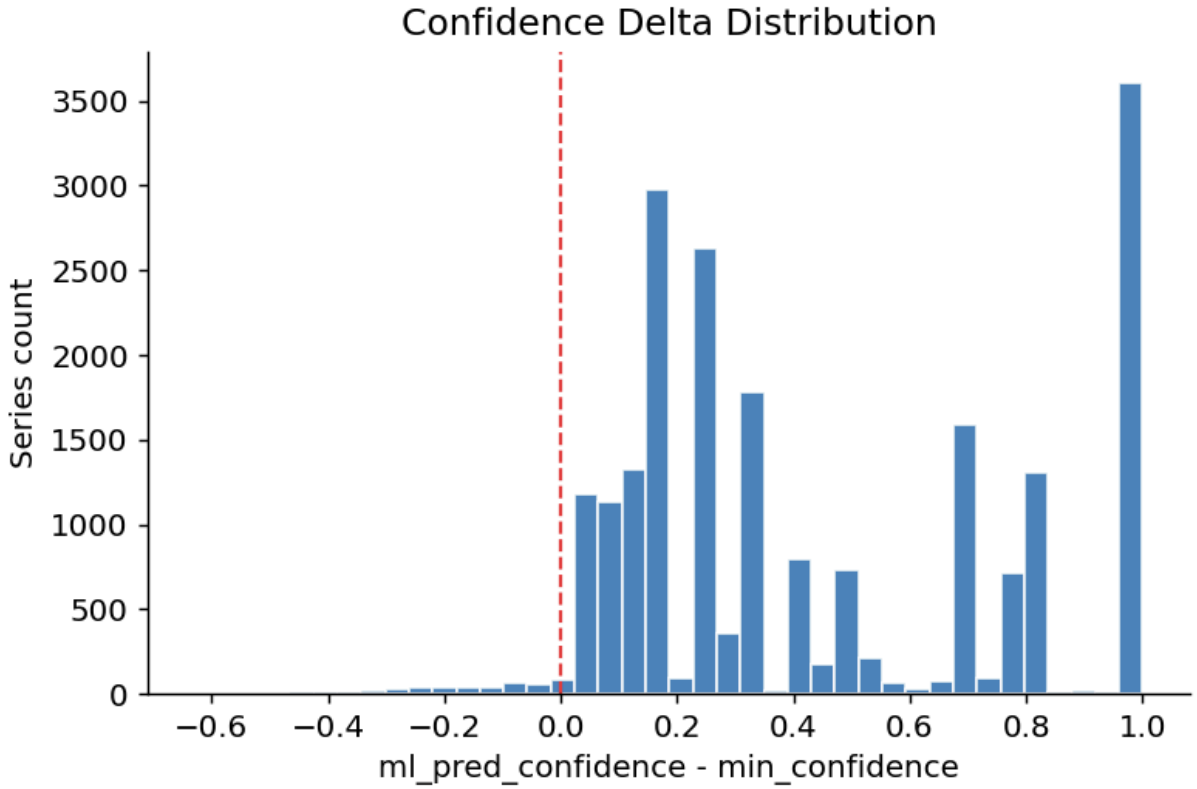
In [32]:

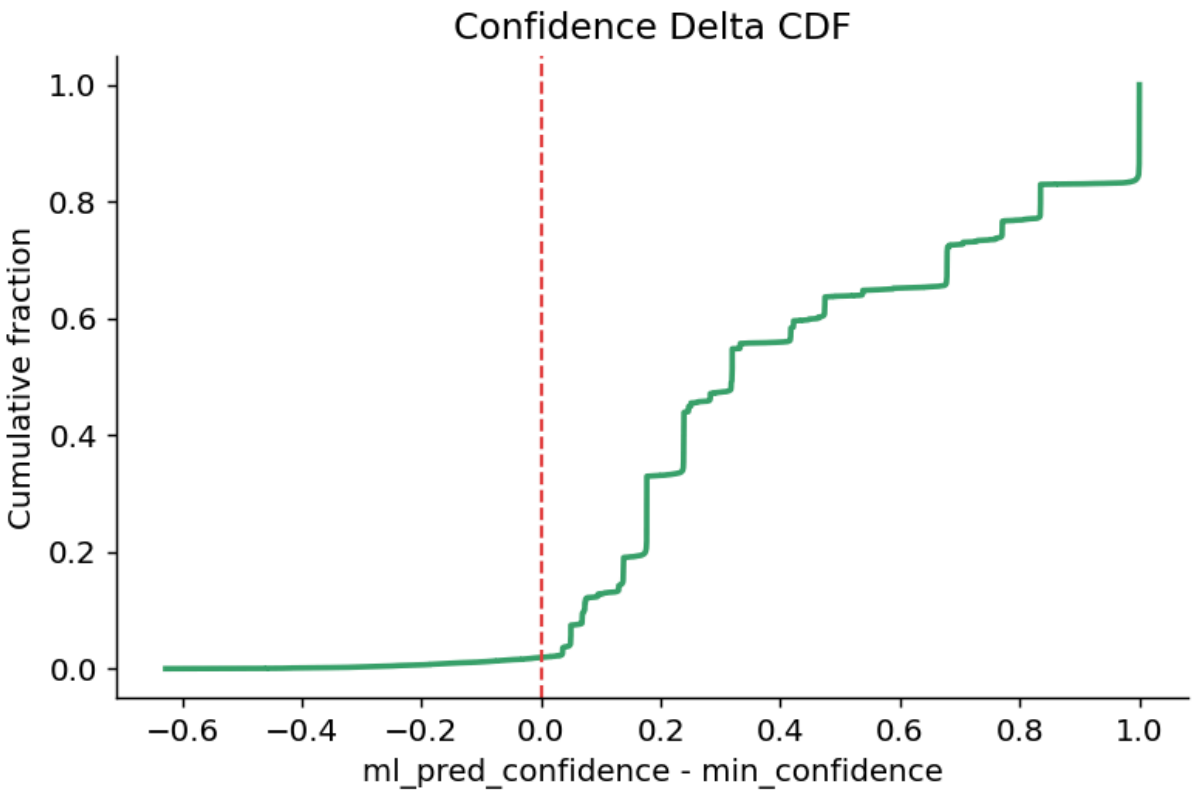
```
# Confidence-delta distribution and data-driven blocks
delta_desc = comp_df["confidence_delta"].describe(percentiles=[0.05, 0.10, 0
delta_desc["positive_rate"] = (comp_df["confidence_delta"] > 0).mean()
delta_desc["negative_rate"] = (comp_df["confidence_delta"] < 0).mean()
delta_desc["zero_rate"] = (comp_df["confidence_delta"] == 0).mean()

# Shared arrays
delta_vals = comp_df["confidence_delta"].to_numpy()
delta_sorted = np.sort(delta_vals)
cdf_y = np.arange(1, len(delta_sorted) + 1) / len(delta_sorted)

_chg = comp_df.loc[comp_df["label_changed"], "confidence_delta"]
_same = comp_df.loc[~comp_df["label_changed"], "confidence_delta"]

# ------------------------------------------------------------
# Combined 3-panel figure (kept as `fig` for downstream save)
# ------------------------------------------------------------
fig, axes = plt.subplots(1, 3, figsize=(16, 4.2))

# Histogram
axes[0].hist(delta_vals, bins=40, color="#2B6CB0", alpha=0.85, edgecolor="wh
axes[0].axvline(0, color="#E53E3E", linestyle="--", linewidth=1.2)
axes[0].set_title("Confidence Delta Distribution")
axes[0].set_xlabel("ml_pred_confidence - min_confidence")
axes[0].set_ylabel("Series count")

# Empirical CDF
axes[1].plot(delta_sorted, cdf_y, color="#38A169", linewidth=2)
axes[1].axvline(0, color="#E53E3E", linestyle="--", linewidth=1.2)
axes[1].set_title("Confidence Delta CDF")
axes[1].set_xlabel("confidence_delta")
axes[1].set_ylabel("Cumulative fraction")

# Changed vs unchanged boxplot
axes[2].boxplot([_same, _chg], tick_labels=["Label unchanged", "Label change
axes[2].axhline(0, color="#E53E3E", linestyle="--", linewidth=1.2)
axes[2].set_title("Confidence Delta by Label Change")
axes[2].set_ylabel("confidence_delta")

plt.tight_layout()
plt.show()

# ------------------------------------------------------------
# Standalone figures (for presentation reuse)
# ------------------------------------------------------------
fig_hist, ax_hist = plt.subplots(figsize=(6.2, 4.2))
ax_hist.hist(delta_vals, bins=40, color="#2B6CB0", alpha=0.85, edgecolor="wh
ax_hist.axvline(0, color="#E53E3E", linestyle="--", linewidth=1.2)
ax_hist.set_title("Confidence Delta Distribution")
ax_hist.set_xlabel("ml_pred_confidence - min_confidence")
ax_hist.set_ylabel("Series count")
plt.tight_layout()
plt.show()

fig_cdf, ax_cdf = plt.subplots(figsize=(6.2, 4.2))
```

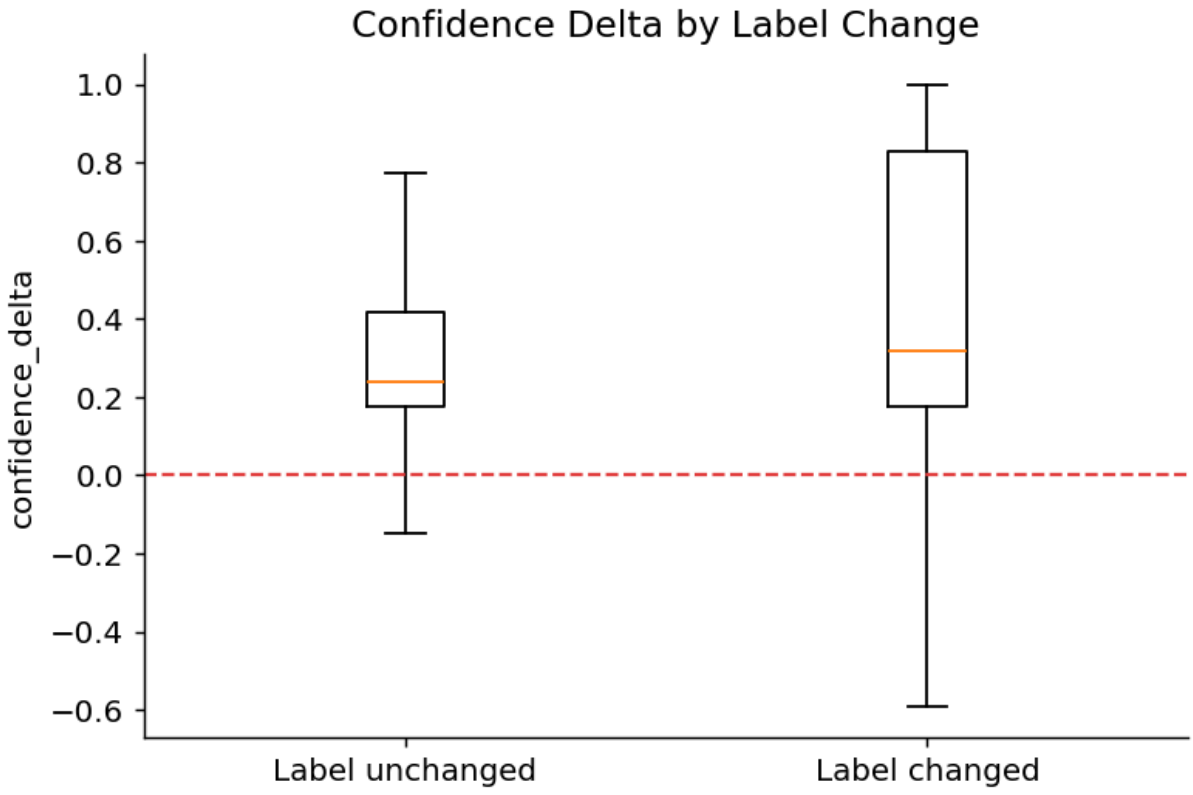
```
ax_cdf.plot(delta_sorted, cdf_y, color="#38A169", linewidth=2)
ax_cdf.axvline(0, color="#E53E3E", linestyle="--", linewidth=1.2)
ax_cdf.set_title("Confidence Delta CDF")
ax_cdf.set_xlabel("ml_pred_confidence - min_confidence")
ax_cdf.set_ylabel("Cumulative fraction")
plt.tight_layout()
plt.show()

fig_box, ax_box = plt.subplots(figsize=(6.2, 4.2))
ax_box.boxplot([_same, _chg], tick_labels=["Label unchanged", "Label changed
ax_box.axhline(0, color="#E53E3E", linestyle="--", linewidth=1.2)
ax_box.set_title("Confidence Delta by Label Change")
ax_box.set_ylabel("confidence_delta")
plt.tight_layout()
plt.show()

# Quantile blocks (adaptive if duplicate edges occur)
n_bins_target = 6
n_unique = int(comp_df["confidence_delta"].nunique())
n_bins = max(2, min(n_bins_target, n_unique))
comp_df["delta_block"] = pd.qcut(comp_df["confidence_delta"], q=n_bins, dupl

block_summary = (
    comp_df.groupby("delta_block", observed=True)
    .agg(
        rows=("series_id", "count"),
        changed_rows=("label_changed", "sum"),
        changed_rate=("label_changed", "mean"),
        delta_median=("confidence_delta", "median"),
        delta_q25=("confidence_delta", lambda s: s.quantile(0.25)),
        delta_q75=("confidence_delta", lambda s: s.quantile(0.75)),
        delta_mean=("confidence_delta", "mean"),
        delta_min=("confidence_delta", "min"),
        delta_max=("confidence_delta", "max"),
    )
    .reset_index()
    .sort_values("delta_median")
)

print("Confidence-delta summary:")
display(delta_desc.style.format({
    "mean": "{:.4f}", "std": "{:.4f}", "min": "{:.4f}", "5%": "{:.4f}",
    "10%": "{:.4f}", "25%": "{:.4f}", "50%": "{:.4f}", "75%": "{:.4f}",
    "90%": "{:.4f}", "95%": "{:.4f}", "max": "{:.4f}",
    "positive_rate": "{:.2%}", "negative_rate": "{:.2%}", "zero_rate": "{:.2
}))

print("\nData-driven confidence blocks (quantile bins):")
display(block_summary.style.format({
    "changed_rate": "{:.2%}",
    "delta_median": "{:.4f}",
    "delta_q25": "{:.4f}",
    "delta_q75": "{:.4f}",
    "delta_mean": "{:.4f}",
    "delta_min": "{:.4f}",
```

```
    "delta_max": "{:.4f}",
}))
```


Confidence Delta Distribution
Series count
ml_pred_confidence - min_confidence
Confidence Delta CDF
Cumulative fraction
confidence_delta
Confidence Delta by Label Change
confidence_delta
Label unchanged
Label changed


Confidence Delta Distribution
Series count
ml_pred_confidence - min_confidence





Confidence-delta summary:

	count	mean	std	min	5%	10%	25%	50%
confidence_delta	21395.000000	0.4449	0.3420	-0.6273	0.0497	0.0726	0.1767	0.3193

Data-driven confidence blocks (quantile bins):

	delta_block	rows	changed_rows	changed_rate	delta_median	delta_q25	delta_q7
0	(-0.628, 0.137]	3566	3015	84.55%	0.0688	0.0489	0.094
1	(0.137, 0.221]	3566	2901	81.35%	0.1767	0.1760	0.176
2	(0.221, 0.319]	3566	2977	83.48%	0.2384	0.2383	0.278
3	(0.319, 0.678]	3566	3190	89.46%	0.4172	0.3195	0.474
4	(0.678, 0.983]	3565	3105	87.10%	0.7708	0.6785	0.834
5	(0.983, 1.0]	3566	3534	99.10%	0.9999	0.9998	1.000

In [30]:
```
# Label-change transitions and artifact export
transition_counts = (
    comp_df[comp_df["label_changed"]]
    .groupby(["heuristic_joint_label", "ml_joint_label"], observed=True)
    .size()
    .rename("rows")
    .reset_index()
    .sort_values("rows", ascending=False)
)
top_transitions = transition_counts.head(20).copy()

group_stats = (
    comp_df.groupby("label_changed", observed=True)["confidence_delta"]
    .agg(["count", "mean", "median", "std", "min", "max"])
    .reset_index()
)
group_stats["label_changed"] = group_stats["label_changed"].map({False: "unc

analysis_plot_dir = PROJECT_DIR / "outputs" / "plots" / "analysis"
analysis_plot_dir.mkdir(parents=True, exist_ok=True)

confidence_summary_path = ANALYSIS_DATA_DIR / "ml_vs_heuristic_confidence_de
block_summary_path = ANALYSIS_DATA_DIR / "ml_vs_heuristic_delta_blocks_summa
transition_path = ANALYSIS_DATA_DIR / "ml_vs_heuristic_label_change_transiti
group_stats_path = ANALYSIS_DATA_DIR / "ml_vs_heuristic_label_change_group_s
headline_path = ANALYSIS_DATA_DIR / "ml_vs_heuristic_headline_metrics.csv"
figure_path = analysis_plot_dir / "ml_vs_heuristic_confidence_delta_overview

delta_desc.to_csv(confidence_summary_path, index=False)
block_summary.to_csv(block_summary_path, index=False)
transition_counts.to_csv(transition_path, index=False)
group_stats.to_csv(group_stats_path, index=False)
headline.to_csv(headline_path, index=False)
fig.savefig(figure_path, bbox_inches="tight")
plt.close(fig)

print("Top heuristic -> ML label transitions (changed rows):")
display(top_transitions)
```

```
print("\nChanged vs unchanged confidence-delta stats:")
display(group_stats.style.format({
    "mean": "{:.4f}",
    "median": "{:.4f}",
    "std": "{:.4f}",
    "min": "{:.4f}",
    "max": "{:.4f}",
}))

print("\nSaved artifacts:")
print(f"- {headline_path}")
print(f"- {confidence_summary_path}")
print(f"- {block_summary_path}")
print(f"- {transition_path}")
print(f"- {group_stats_path}")
print(f"- {figure_path}")
```

```
Top heuristic -> ML label transitions (changed rows):
```

	heuristic_joint_label	ml_joint_label	rows
118	dwi::dwi	anat::T1w	898
41	anat::T1w	dwi::dwi	699
246	localizer::localizer	dwi::dwi	656
123	dwi::dwi	dwi::dwiParamMap	615
130	dwi::dwi	localizer::localizer	602
157	dwi::dwiParamMap	dwi::dwi	545
122	dwi::dwi	dwi::dwiOtherDerived	441
242	localizer::localizer	anat::T1w	402
185	dwi::unknown	dwi::dwi	386
141	dwi::dwiOtherDerived	dwi::dwi	369
153	dwi::dwiParamMap	anat::T1w	355
50	anat::T1w	localizer::localizer	350
132	dwi::dwi	perf::dsc	338
43	anat::T1w	dwi::dwiParamMap	327
6	anat::FLAIR	dwi::dwi	321
115	dwi::dwi	anat::FLAIR	299
277	perf::dsc	dwi::dwi	292
248	localizer::localizer	dwi::dwiParamMap	289
104	anat::unknown	dwi::dwi	285
181	dwi::unknown	anat::T1w	260

```
Changed vs unchanged confidence-delta stats:
```

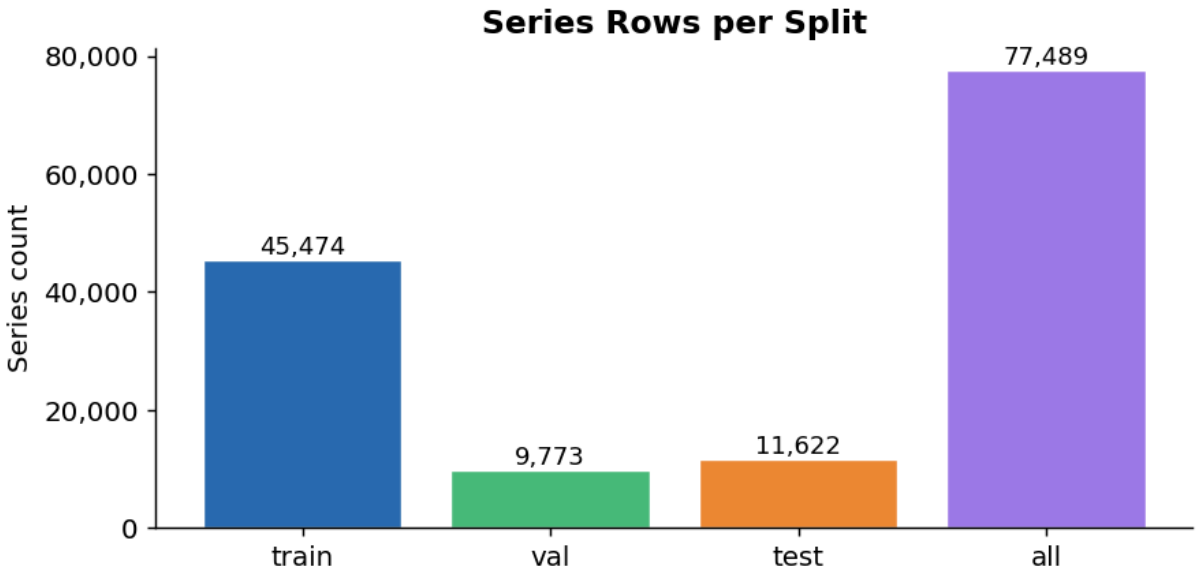
	label_changed	count	mean	median	std	min	max
0	unchanged	2673	0.3106	0.2383	0.2563	-0.6273	1.0000
1	changed	18722	0.4641	0.3195	0.3483	-0.5927	1.0000

```
Saved artifacts:
- /Users/cmokashi/Documents/GitHub/find_BIDS/outputs/analysis_data/ml_vs_heu
ristic_headline_metrics.csv
- /Users/cmokashi/Documents/GitHub/find_BIDS/outputs/analysis_data/ml_vs_heu
ristic_confidence_delta_summary.csv
- /Users/cmokashi/Documents/GitHub/find_BIDS/outputs/analysis_data/ml_vs_heu
ristic_delta_blocks_summary.csv
- /Users/cmokashi/Documents/GitHub/find_BIDS/outputs/analysis_data/ml_vs_heu
ristic_label_change_transitions.csv
- /Users/cmokashi/Documents/GitHub/find_BIDS/outputs/analysis_data/ml_vs_heu
ristic_label_change_group_stats.csv
- /Users/cmokashi/Documents/GitHub/find_BIDS/outputs/plots/analysis/ml_vs_he
uristic_confidence_delta_overview.png
```

Part 3 — Dataset Overview

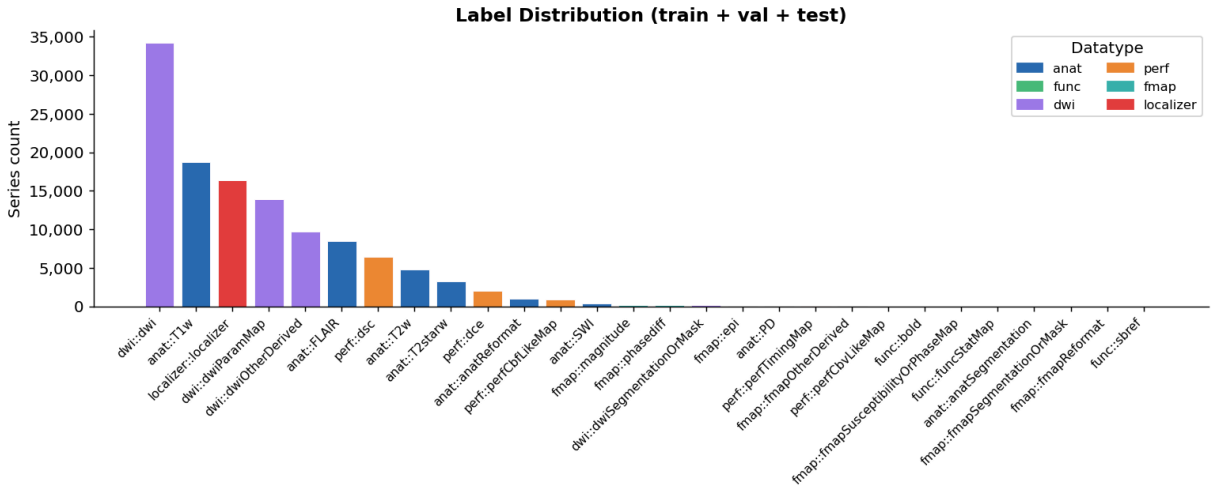
In [4]:

```python
# Split size bar chart  ("all" = full labeled pool used for the final produc
_split_order  = ["train", "val", "test", "all"]
_split_colors = {"train": "#2B6CB0", "val": "#48BB78", "test": "#ED8936", "a

split_plot_df = split_summary.copy()
split_plot_df["split"] = pd.Categorical(split_plot_df["split"], categories=_
split_plot_df = split_plot_df.sort_values("split").dropna(subset=["split"])

fig, ax = plt.subplots(figsize=(7, 3.5))
colors = [_split_colors.get(str(s), "#718096") for s in split_plot_df["split
bars = ax.bar(split_plot_df["split"].astype(str), split_plot_df["rows"], col
for bar, val in zip(bars, split_plot_df["rows"]):
    ax.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 80, f"{int
            ha="center", va="bottom", fontsize=10)
ax.set_title("Series Rows per Split", fontweight="bold")
ax.set_ylabel("Series count")
ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda x, _: f"{int(x):,}"))
plt.tight_layout()
plt.show()
plt.close(fig)
```



In [ ]:

In [5]:
```
# Label distribution across train + val + test
vc = labeled_df["joint_label"].value_counts()
label_counts = pd.DataFrame({"joint_label": vc.index, "count": vc.values})
label_counts["datatype"] = label_counts["joint_label"].str.split("::").str[0
label_counts = label_counts.sort_values("count", ascending=False)

bar_colors = [_family_palette.get(d, "#A0AEC0") for d in label_counts["datat

fig, ax = plt.subplots(figsize=(12, 5))
ax.bar(range(len(label_counts)), label_counts["count"], color=bar_colors, ec
ax.set_xticks(range(len(label_counts)))
ax.set_xticklabels(label_counts["joint_label"], rotation=45, ha="right", for
ax.set_ylabel("Series count")
ax.set_title("Label Distribution (train + val + test)", fontweight="bold")
ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda x, _: f"{int(x):,}"))
legend_patches = [
    mpatches.Patch(color=c, label=f)
    for f, c in _family_palette.items()
    if f in label_counts["datatype"].values
]
ax.legend(handles=legend_patches, title="Datatype", loc="upper right", fonts
plt.tight_layout()
plt.show()
plt.close(fig)

print(f"Classes present: {len(label_counts)}")
display(label_counts.head(12))
```



Classes present: 28

	joint_label	count	datatype
0	dwi::dwi	34184	dwi
1	anat::T1w	18710	anat
2	localizer::localizer	16340	localizer
3	dwi::dwiParamMap	13882	dwi
4	dwi::dwiOtherDerived	9672	dwi
5	anat::FLAIR	8505	anat
6	perf::dsc	6453	perf
7	anat::T2w	4772	anat
8	anat::T2starw	3307	anat
9	perf::dce	2060	perf
10	anat::anatReformat	958	anat
11	perf::perfCbfLikeMap	851	perf

In [6]:
```python
# Per-dataset breakdown
if "dataset" in analysis_df.columns:
    dataset_summary = (
        analysis_df.groupby("dataset")
        .agg(
            total_series=("series_id", "count"),
            unique_subjects=("subject_id", "nunique"),
            unknown_series=("is_heuristic_unknown", "sum"),
        )
        .reset_index()
        .sort_values("total_series", ascending=False)
    )
    dataset_summary["unknown_pct"] = (
        dataset_summary["unknown_series"] / dataset_summary["total_series"]
    ).round(1)
```

```
    print("Per-dataset summary:")
    display(dataset_summary)
```

```
Per-dataset summary:
```

	dataset	total_series	unique_subjects	unknown_series	unknown_pct
0	proactive	79153	174	11403	14.4000
1	qiac	65205	516	11957	18.3000

Part 4 — Heuristic Confidence Analysis

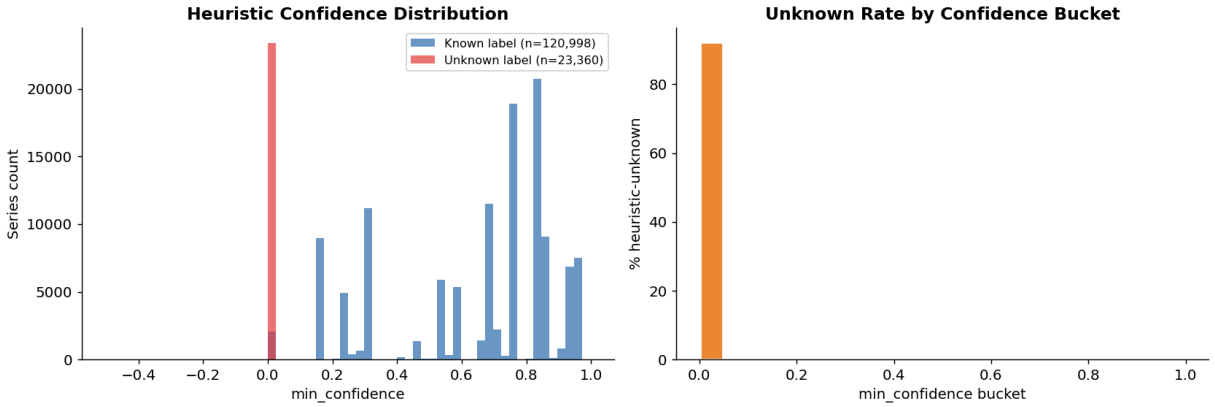
Before the ML model runs, the heuristic engine assigns a `min_confidence` score to every series. The plots below show:

- How confidence distributes across labeled vs. unknown-label series
- How the unknown rate changes across confidence buckets (lower confidence → more unknowns)

Series where `min_confidence` is near zero or either label part is `unknown` are the **ML candidates** — they are excluded from supervised training and instead scored by the fitted model.

In [7]:
```
fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))

# Left: confidence histogram, known vs unknown
ax = axes[0]
conf_known   = labeled_df["min_confidence"].dropna()
conf_unknown = excluded_df["min_confidence"].dropna()
ax.hist(conf_known,   bins=40, alpha=0.7, color="#2B6CB0", label=f"Known lab
ax.hist(conf_unknown, bins=40, alpha=0.7, color="#E53E3E", label=f"Unknown l
ax.set_xlabel("min_confidence")
ax.set_ylabel("Series count")
ax.set_title("Heuristic Confidence Distribution", fontweight="bold")
ax.legend(fontsize=9)

# Right: unknown rate per confidence bucket
ax2 = axes[1]
conf_series  = analysis_df["min_confidence"].dropna()
unk_series   = analysis_df.loc[analysis_df["min_confidence"].notna(), "is_he
bins         = np.linspace(0, 1, 21)
bin_idx      = np.digitize(conf_series, bins) - 1
bin_rates, bin_centers = [], []
for b in range(len(bins) - 1):
    mask = bin_idx == b
    if mask.sum() > 0:
        bin_rates.append(unk_series[mask].mean() * 100)
        bin_centers.append((bins[b] + bins[b + 1]) / 2)
ax2.bar(bin_centers, bin_rates, width=0.045, color="#ED8936", edgecolor="whi
ax2.set_xlabel("min_confidence bucket")
```

```
ax2.set_ylabel("% heuristic-unknown")
ax2.set_title("Unknown Rate by Confidence Bucket", fontweight="bold")

plt.tight_layout()
plt.show()
plt.close(fig)

total     = len(analysis_df)
n_unknown = int(analysis_df["is_heuristic_unknown"].sum())
print(f"Heuristic-unknown series: {n_unknown:,} / {total:,}  ({n_unknown/tot
print(f"Median confidence (known): {conf_known.median():.4f}")
print(f"Median confidence (unknown): {conf_unknown.median():.4f}")
```




```
Heuristic-unknown series: 23,360 / 144,358  (16.2%)
Median confidence (known): 0.7616
Median confidence (unknown): 0.0000
```

Part 5 — ML Model Configuration

Setting	Value
Model family	`XGBClassifier`
Configuration	`xgboost_depth6` (350 estimators, max_depth=6, learning_rate=0.08)
Target	`inferred_joint_label` (`dtype::suffix`)
Training regime	Confidence-weighted (`min_confidence` as `sample_weight`)
Weight floor	0.30 — low-confidence samples get at least 30% weight
Unknown exclusion	`drop_unknown_in_train_val=True` — unknown-label rows held out of train+val
Split fractions	70% train / 15% val / 15% test (`random_state=42`)
Feature types	TF-IDF text, keyword flags, engineered numerics, missing indicators

Why confidence weighting? Series where the heuristic engine is uncertain carry noisier labels. Downweighting these rows (without discarding them) keeps the model from over-fitting to low-quality training signal while still benefiting from the additional data volume.

In [8]:
```
display(final_metrics[[
    "target", "split", "rows", "classes_in_train", "weighted_training"
]])
```

	target	split	rows	classes_in_train	weighted_training
0	inferred_joint_label	test	11622	28	True
1	inferred_joint_label	val	9773	28	True

Part 6 — Overall Performance

Metric Summary (Validation vs Test)

Metric	Description	Validation	Test
Accuracy	Overall fraction of correctly classified series	**0.9958**	**0.9951**
Balanced Accuracy	Mean recall across classes (treats common and rare classes equally)	**0.8464**	**0.7983**
Macro F1	Unweighted average F1 across classes (sensitive to rare-class performance)	**0.8383**	**0.7842**
Macro OvR AUC	One-vs-rest ROC AUC averaged across classes	**0.9998**	**0.9993**
Log Loss *(lower is better)*	Penalized probability error; lower means better-calibrated confident predictions	**0.0184**	**0.0159**
Top-2 Accuracy	Fraction where the true class is within the top 2 predicted classes	**0.9989**	**0.9989**
Top-3 Accuracy	Fraction where the true class is within the top 3 predicted classes	**0.9995**	**0.9993**

Interpretation

The model delivers **very strong overall performance** on both splits (accuracy ≈ 99.5%, macro AUC ≈ 0.999).
The lower **macro F1** and **balanced accuracy** indicate that remaining errors are concentrated in a small number of **rare classes**, rather than the frequent classes.

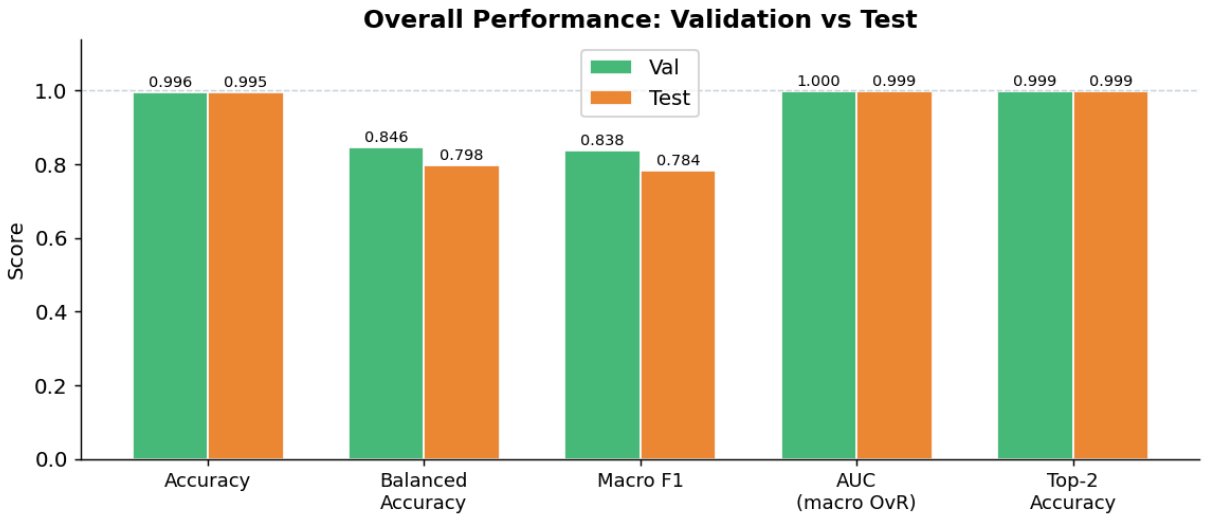
In [10]:
```
# Headline metrics table
_metric_cols = [
    "split", "accuracy", "balanced_accuracy", "f1_macro",
    "auc_macro_ovr", "log_loss", "top2_accuracy", "top3_accuracy",
]
headline = final_metrics[_metric_cols].copy()
headline["split"] = pd.Categorical(headline["split"], categories=["val", "te
headline = headline.sort_values("split")
```

```
display(headline.style
    .format({c: "{:.4f}" for c in _metric_cols[1:]})
    .background_gradient(subset=["accuracy", "f1_macro", "auc_macro_ovr"], c
    .set_caption("Final Model — Validation and Test Metrics")
)
```

Final Model — Validation and Test Metrics

	split	accuracy	balanced_accuracy	f1_macro	auc_macro_ovr	log_loss	top2_accur
1	val	0.9958	0.8464	0.8383	0.9998	0.0184	0.99
0	test	0.9951	0.7983	0.7842	0.9993	0.0159	0.99

In [11]:
```
# Grouped bar chart: val vs test on key metrics
_plot_metrics  = ["accuracy", "balanced_accuracy", "f1_macro", "auc_macro_ov
_metric_labels = ["Accuracy", "Balanced\nAccuracy", "Macro F1", "AUC\n(macro

val_row  = headline[headline["split"] == "val"].iloc[0]
test_row = headline[headline["split"] == "test"].iloc[0]

x, width = np.arange(len(_plot_metrics)), 0.35
fig, ax = plt.subplots(figsize=(9, 4))
ax.bar(x - width/2, [val_row[m]  for m in _plot_metrics], width, label="Val"
ax.bar(x + width/2, [test_row[m] for m in _plot_metrics], width, label="Test
for xi, m in enumerate(_plot_metrics):
    vv, tv = val_row[m], test_row[m]
    ax.text(xi - width/2, vv + 0.006, f"{vv:.3f}", ha="center", va="bottom",
    ax.text(xi + width/2, tv + 0.006, f"{tv:.3f}", ha="center", va="bottom",
ax.set_xticks(x)
ax.set_xticklabels(_metric_labels, fontsize=10)
ax.set_ylim(0, 1.14)
ax.set_ylabel("Score")
ax.set_title("Overall Performance: Validation vs Test", fontweight="bold")
ax.legend()
ax.axhline(1.0, color="#CBD5E0", linewidth=0.8, linestyle="--")
plt.tight_layout()
plt.show()
plt.close(fig)
```



Part 7 — Per-Class Performance

The model predicts **28 distinct `dtype::suffix` classes** on the training set; the test set contains **24 classes** (four rare classes are absent after stratified splitting).

The breakdown below shows precision, recall, F1, and sample count for every class present in the test fold.

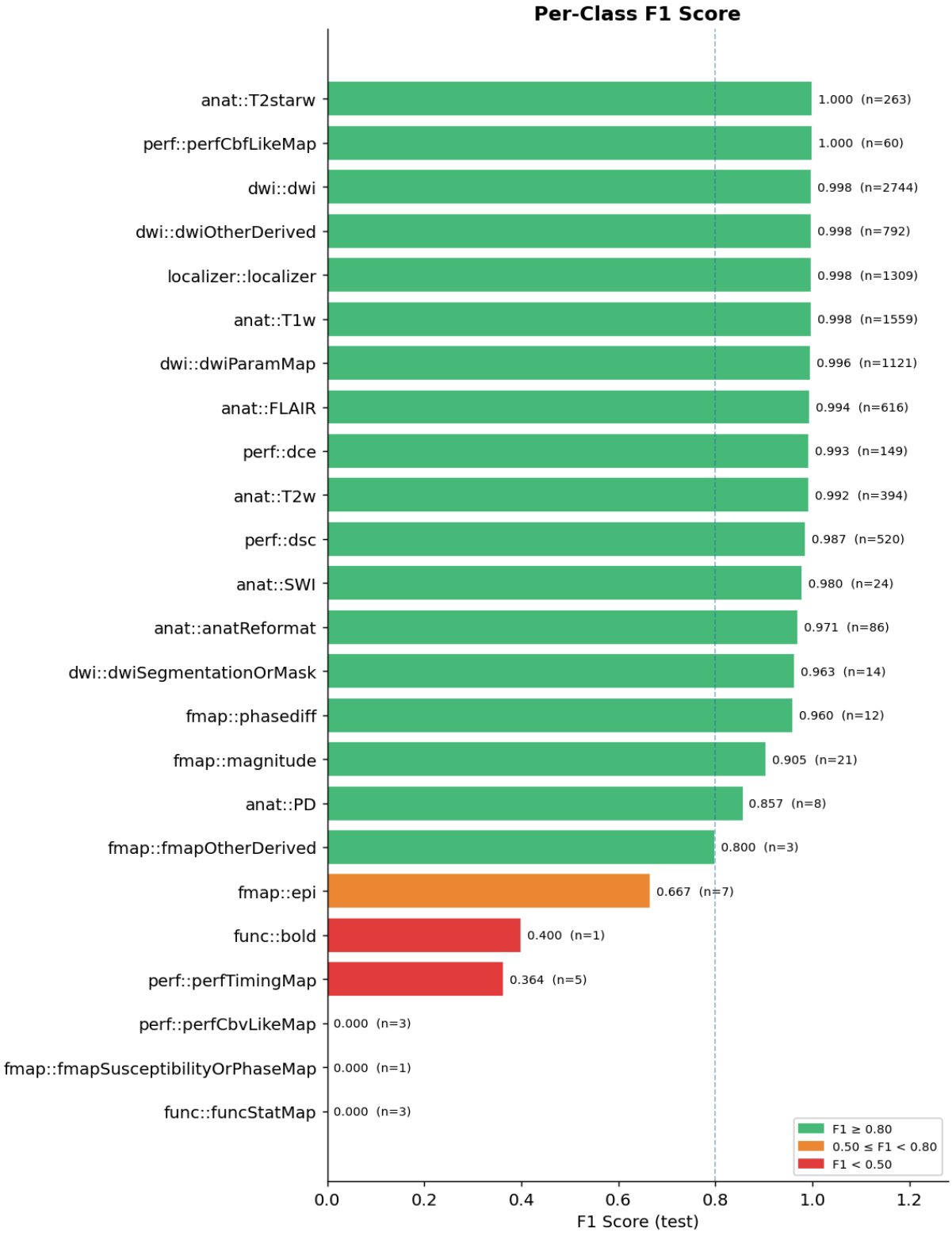
In [12]:
```
# Per-class table sorted by F1
display_report = class_report.sort_values("f1-score", ascending=False)[
    ["label", "datatype", "precision", "recall", "f1-score", "support"]
].copy()
display(display_report.style
    .format({"precision": "{:.3f}", "recall": "{:.3f}", "f1-score": "{:.3f}"
    .background_gradient(subset=["f1-score"], cmap="RdYlGn")
    .set_caption("Per-Class Classification Report (test split, sorted by F1)
)
```

Per-Class Classification Report (test split, sorted by F1)

	label	datatype	precision	recall	f1-score	support
21	perf::perfCbfLikeMap	perf	1.000	1.000	1.000	60
4	anat::T2starw	anat	1.000	1.000	1.000	263
7	dwi::dwi	dwi	0.998	0.999	0.998	2744
8	dwi::dwiOtherDerived	dwi	0.996	1.000	0.998	792
18	localizer::localizer	localizer	1.000	0.996	0.998	1309
3	anat::T1w	anat	0.998	0.998	0.998	1559
9	dwi::dwiParamMap	dwi	0.996	0.997	0.996	1121
0	anat::FLAIR	anat	0.994	0.995	0.994	616
19	perf::dce	perf	0.993	0.993	0.993	149
5	anat::T2w	anat	0.992	0.992	0.992	394
20	perf::dsc	perf	0.983	0.990	0.987	520
2	anat::SWI	anat	0.960	1.000	0.980	24
6	anat::anatReformat	anat	0.966	0.977	0.971	86
10	dwi::dwiSegmentationOrMask	dwi	1.000	0.929	0.963	14
15	fmap::phasediff	fmap	0.923	1.000	0.960	12
14	fmap::magnitude	fmap	0.905	0.905	0.905	21
1	anat::PD	anat	1.000	0.750	0.857	8
12	fmap::fmapOtherDerived	fmap	1.000	0.667	0.800	3
11	fmap::epi	fmap	0.800	0.571	0.667	7
16	func::bold	func	0.250	1.000	0.400	1
23	perf::perfTimingMap	perf	0.333	0.400	0.364	5
17	func::funcStatMap	func	0.000	0.000	0.000	3
13	fmap::fmapSusceptibilityOrPhaseMap	fmap	0.000	0.000	0.000	1
22	perf::perfCbvLikeMap	perf	0.000	0.000	0.000	3

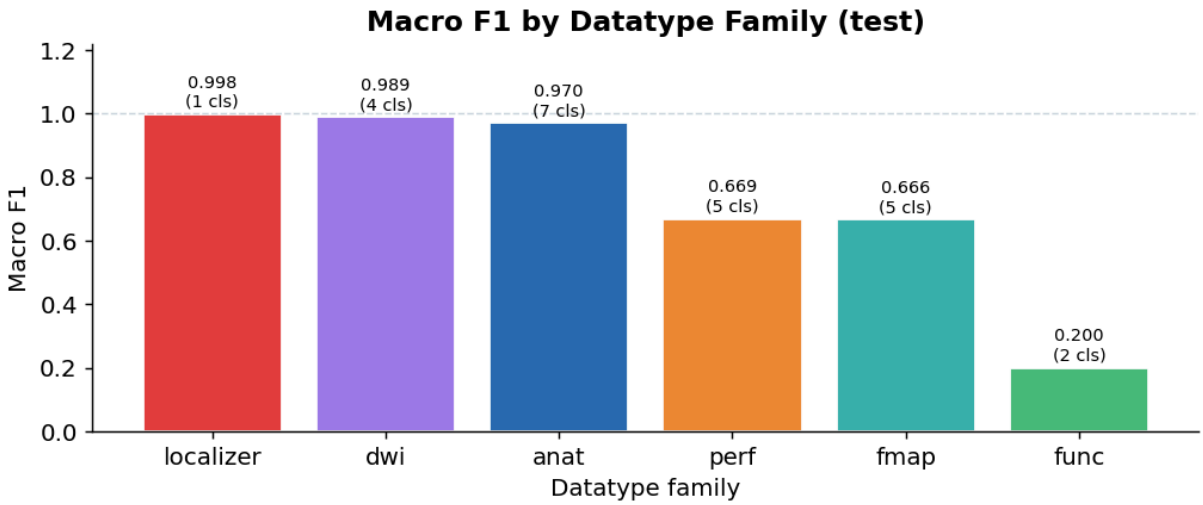
In [13]:
```
# F1 horizontal bar chart
sorted_report = class_report.sort_values("f1-score")
bar_colors_f1 = [
    "#E53E3E" if f < 0.50 else ("#ED8936" if f < 0.80 else "#48BB78")
    for f in sorted_report["f1-score"]
]
fig, ax = plt.subplots(figsize=(9, len(sorted_report) * 0.44 + 1))
bars = ax.barh(sorted_report["label"], sorted_report["f1-score"],
               color=bar_colors_f1, edgecolor="white")
for bar, (_, row) in zip(bars, sorted_report.iterrows()):
```

```
    ax.text(
        min(row["f1-score"] + 0.012, 1.02),
        bar.get_y() + bar.get_height() / 2,
        f"{row['f1-score']:.3f}  (n={int(row['support'])})",
        va="center", fontsize=8,
    )
ax.set_xlim(0, 1.28)
ax.set_xlabel("F1 Score (test)")
ax.set_title("Per-Class F1 Score", fontweight="bold")
ax.axvline(0.80, color="#2B6CB0", linewidth=0.9, linestyle="--", alpha=0.5)
legend_patches = [
    mpatches.Patch(color="#48BB78", label="F1 ≥ 0.80"),
    mpatches.Patch(color="#ED8936", label="0.50 ≤ F1 < 0.80"),
    mpatches.Patch(color="#E53E3E", label="F1 < 0.50"),
]
ax.legend(handles=legend_patches, fontsize=8, loc="lower right")
plt.tight_layout()
plt.show()
plt.close(fig)
```



In [14]:
```python
# Macro F1 by datatype family
family_f1 = (
    class_report.groupby("datatype")
    .agg(
        macro_f1=("f1-score", "mean"),
        n_classes=("label", "count"),
        total_support=("support", "sum"),
    )
    .reset_index()
```

```
    .sort_values("macro_f1", ascending=False)
)

fam_colors = [_family_palette.get(d, "#A0AEC0") for d in family_f1["datatype
fig, ax = plt.subplots(figsize=(8, 3.5))
ax.bar(family_f1["datatype"], family_f1["macro_f1"], color=fam_colors, edgec
for i, (_, row) in enumerate(family_f1.iterrows()):
    ax.text(i, row["macro_f1"] + 0.012,
            f"{row['macro_f1']:.3f}\n({int(row['n_classes'])} cls)",
            ha="center", va="bottom", fontsize=8)
ax.set_ylim(0, 1.22)
ax.set_xlabel("Datatype family")
ax.set_ylabel("Macro F1")
ax.set_title("Macro F1 by Datatype Family (test)", fontweight="bold")
ax.axhline(1.0, color="#CBD5E0", linewidth=0.8, linestyle="--")
plt.tight_layout()
plt.show()
plt.close(fig)

display(family_f1)
```



	datatype	macro_f1	n_classes	total_support
4	localizer	0.9981	1	1309
1	dwi	0.9890	4	4671
0	anat	0.9704	7	2950
5	perf	0.6687	5	737
2	fmap	0.6663	5	44
3	func	0.2000	2	4

Part 8 — Weak and Low-Support Classes

Some classes have very few test samples (`support ≤ 5`) or F1 below 0.50. These fall into two categories:

1. **Rare acquisitions** — functionally valid BIDS types that are simply uncommon in our dataset (e.g., `func::funcStatMap` , `perf::perfCbvLikeMap`). More training data would close the gap.
2. **Ambiguous sequences** — scan types that share DICOM characteristics with more common types, creating genuine classification difficulty even with more data.

The table below lists every class with `f1-score < 0.80` or `support ≤ 5` in the test set.

In [15]:
```
weak_classes = class_report[
    (class_report["f1-score"] < 0.80) | (class_report["support"] <= 5)
].sort_values("f1-score")[["label", "datatype", "precision", "recall", "f1-s

print(f"Classes with F1 < 0.80 or support ≤ 5: {len(weak_classes)} / {len(cl
display(weak_classes.style
    .format({"precision": "{:.3f}", "recall": "{:.3f}", "f1-score": "{:.3f}"
    .background_gradient(subset=["f1-score"], cmap="RdYlGn")
    .set_caption("Weak / Low-Support Classes")
)

# Val vs test comparison for weak classes
_val_report = report_val[~report_val["label"].isin(_summary_labels)].copy()
_val_report["support"] = _val_report["support"].astype(int)
weak_labels = weak_classes["label"].tolist()
comparison = pd.merge(
    class_report[class_report["label"].isin(weak_labels)][["label", "f1-scor
        columns={"f1-score": "f1_test", "support": "support_test"}
    ),
    _val_report[_val_report["label"].isin(weak_labels)][["label", "f1-score"
        columns={"f1-score": "f1_val", "support": "support_val"}
    ),
    on="label", how="outer",
).sort_values("f1_test")
print("\nWeak classes — val vs test F1:")
display(comparison)
```

```
Classes with F1 < 0.80 or support ≤ 5: 7 / 24
```

Weak / Low-Support Classes

	label	datatype	precision	recall	f1-score	support
13	fmap::fmapSusceptibilityOrPhaseMap	fmap	0.000	0.000	0.000	1
17	func::funcStatMap	func	0.000	0.000	0.000	3
22	perf::perfCbvLikeMap	perf	0.000	0.000	0.000	3
23	perf::perfTimingMap	perf	0.333	0.400	0.364	5
16	func::bold	func	0.250	1.000	0.400	1
11	fmap::epi	fmap	0.800	0.571	0.667	7
12	fmap::fmapOtherDerived	fmap	1.000	0.667	0.800	3

Weak classes — val vs test F1:

	label	f1_test	support_test	f1_val	support_val
2	fmap::fmapSusceptibilityOrPhaseMap	0.0000	1	0.0000	2.0000
4	func::funcStatMap	0.0000	3	NaN	NaN
5	perf::perfCbvLikeMap	0.0000	3	NaN	NaN
6	perf::perfTimingMap	0.3636	5	0.6000	5.0000
3	func::bold	0.4000	1	1.0000	2.0000
0	fmap::epi	0.6667	7	0.3333	5.0000
1	fmap::fmapOtherDerived	0.8000	3	0.5714	2.0000

Part 8B — Weak-Class Misidentification Deep Dive

This section answers:

1. Which low-performing classes are being misidentified as what?
2. Are those errors concentrated in one dominant confusion path or spread out?
3. Are mistakes low-confidence (uncertainty) or high-confidence (calibration risk)?

In [21]:
```
# Build weak-class confusion diagnostics on test split
true_col = "joint_label"
pred_col_export = "ml_pred_joint_label"

# Use only test rows with both true and predicted labels available
test_eval = analysis_df[
    (analysis_df["ml_split"] == "test")
    & analysis_df[true_col].notna()
    & analysis_df[pred_col_export].notna()
].copy()
test_eval["is_correct"] = test_eval[true_col] == test_eval[pred_col_export]

# Recompute weak labels from class_report for reproducibility
```

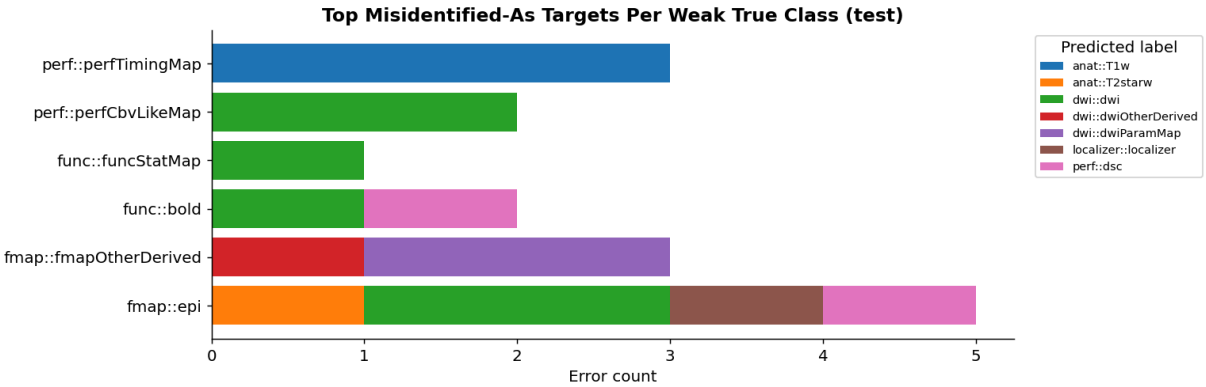
```
weak_labels = set(
    class_report[
        (class_report["f1-score"] < 0.80) | (class_report["support"] <= 5)
    ]["label"].tolist()
)

weak_eval = test_eval[test_eval[true_col].isin(weak_labels)].copy()
weak_errors = weak_eval[~weak_eval["is_correct"]].copy()

confusion_pairs = (
    weak_errors.groupby([true_col, pred_col_export])
    .size()
    .rename("errors")
    .reset_index()
)

true_support = (
    weak_eval.groupby(true_col)
    .size()
    .rename("support")
    .reset_index()
)

weak_confusion_summary = confusion_pairs.merge(true_support, on=true_col, ho
weak_confusion_summary["error_share_of_class"] = (
    weak_confusion_summary["errors"] / weak_confusion_summary["support"]
).fillna(0.0)

weak_confusion_summary = weak_confusion_summary.sort_values(
    [true_col, "errors"], ascending=[True, False]
)

print(f"Weak classes on test: {len(weak_labels)}")
print(f"Weak-class test rows: {len(weak_eval):,}")
print(f"Weak-class test errors: {len(weak_errors):,}")

display(
    weak_confusion_summary[[
        true_col, pred_col_export, "errors", "support", "error_share_of_clas
    ]].head(30)
)
```

```
Weak classes on test: 7
Weak-class test rows: 16
Weak-class test errors: 16
```

	joint_label	ml_pred_joint_label	errors	support	error_share_of_class
1	fmap::epi	dwi::dwi	2	5	0.4000
0	fmap::epi	anat::T2starw	1	5	0.2000
2	fmap::epi	localizer::localizer	1	5	0.2000
3	fmap::epi	perf::dsc	1	5	0.2000
5	fmap::fmapOtherDerived	dwi::dwiParamMap	2	3	0.6667
4	fmap::fmapOtherDerived	dwi::dwiOtherDerived	1	3	0.3333
6	func::bold	dwi::dwi	1	2	0.5000
7	func::bold	perf::dsc	1	2	0.5000
8	func::funcStatMap	dwi::dwi	1	1	1.0000
9	perf::perfCbvLikeMap	dwi::dwi	2	2	1.0000
10	perf::perfTimingMap	anat::T1w	3	3	1.0000

In [22]:
```
# Visualize dominant misidentification pathways per weak class
_topk = 5
plot_df = (
    weak_confusion_summary.sort_values([true_col, "errors"], ascending=[True
    .groupby(true_col)
    .head(_topk)
    .copy()
)

if len(plot_df) == 0:
    print("No weak-class errors found on test split.")
else:
    pivot = plot_df.pivot_table(
        index=true_col,
        columns=pred_col_export,
        values="errors",
        aggfunc="sum",
        fill_value=0,
    )

    fig, ax = plt.subplots(figsize=(12, max(4, 0.6 * len(pivot))))
    left = np.zeros(len(pivot))
    y_pos = np.arange(len(pivot))

    for pred_label in pivot.columns:
        values = pivot[pred_label].values
        ax.barh(y_pos, values, left=left, label=pred_label)
        left += values

    ax.set_yticks(y_pos)
    ax.set_yticklabels(pivot.index)
    ax.set_xlabel("Error count")
    ax.set_title("Top Misidentified-As Targets Per Weak True Class (test)",
    ax.legend(title="Predicted label", bbox_to_anchor=(1.02, 1), loc="upper
```

```
    plt.tight_layout()
    plt.show()
    plt.close(fig)

    top1 = (
        weak_confusion_summary.sort_values([true_col, "errors"], ascending=[
        .groupby(true_col)
        .head(1)
        [[true_col, pred_col_export, "errors", "support", "error_share_of_cl
        .rename(columns={pred_col_export: "top_misidentified_as"})
    )
    print("Top-1 confusion target per weak class:")
    display(top1)
```



Top-1 confusion target per weak class:

	joint_label	top_misidentified_as	errors	support	error_share_of_class
1	fmap::epi	dwi::dwi	2	5	0.4000
5	fmap::fmapOtherDerived	dwi::dwiParamMap	2	3	0.6667
6	func::bold	dwi::dwi	1	2	0.5000
8	func::funcStatMap	dwi::dwi	1	1	1.0000
9	perf::perfCbvLikeMap	dwi::dwi	2	2	1.0000
10	perf::perfTimingMap	anat::T1w	3	3	1.0000

In [23]:
```
# Confidence and support diagnostics for weak classes
weak_eval = weak_eval.copy()
weak_eval["correctness"] = np.where(weak_eval["is_correct"], "correct", "inc

conf_stats = (
    weak_eval.groupby([true_col, "correctness"])["ml_pred_confidence"]
    .agg(["count", "mean", "median"])
    .reset_index()
    .sort_values([true_col, "correctness"])
)

# Per-class support across train/val/test
split_support = (
    analysis_df[
        analysis_df[true_col].isin(weak_labels)
        & analysis_df["ml_split"].isin(["train", "val", "test"])
```

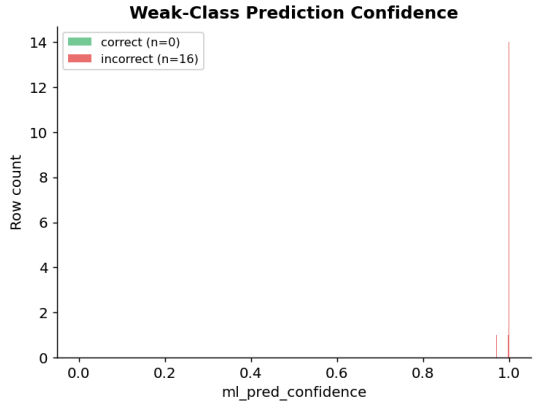
```
    ]
    .groupby([true_col, "ml_split"])
    .size()
    .rename("rows")
    .reset_index()
)

support_wide = split_support.pivot(index=true_col, columns="ml_split", value
for col in ["train", "val", "test"]:
    if col not in support_wide.columns:
        support_wide[col] = 0
support_wide = support_wide[["train", "val", "test"]].reset_index()

print("Confidence summary by weak class and correctness:")
display(conf_stats)

print("Support by split for weak classes:")
display(support_wide)

# Plot confidence distributions: correct vs incorrect for weak classes
fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))

ax = axes[0]
correct_conf = weak_eval.loc[weak_eval["is_correct"], "ml_pred_confidence"].
incorrect_conf = weak_eval.loc[~weak_eval["is_correct"], "ml_pred_confidence
ax.hist(correct_conf, bins=20, alpha=0.75, label=f"correct (n={len(correct_c
ax.hist(incorrect_conf, bins=20, alpha=0.75, label=f"incorrect (n={len(incor
ax.set_title("Weak-Class Prediction Confidence", fontweight="bold")
ax.set_xlabel("ml_pred_confidence")
ax.set_ylabel("Row count")
ax.legend(fontsize=9)

ax2 = axes[1]
error_rate_by_class = (
    weak_eval.groupby(true_col)["is_correct"]
    .apply(lambda s: 1 - s.mean())
    .sort_values(ascending=False)
)
ax2.barh(error_rate_by_class.index, error_rate_by_class.values, color="#ED89
ax2.set_xlabel("Error rate on test")
ax2.set_title("Weak-Class Error Rate", fontweight="bold")

plt.tight_layout()
plt.show()
plt.close(fig)
```

```
Confidence summary by weak class and correctness:
```

	joint_label	correctness	count	mean	median
0	fmap::epi	incorrect	5	0.9938	1.0000
1	fmap::fmapOtherDerived	incorrect	3	0.9999	0.9999
2	func::bold	incorrect	2	0.9989	0.9989
3	func::funcStatMap	incorrect	1	1.0000	1.0000
4	perf::perfCbvLikeMap	incorrect	2	1.0000	1.0000
5	perf::perfTimingMap	incorrect	3	1.0000	1.0000

Support by split for weak classes:

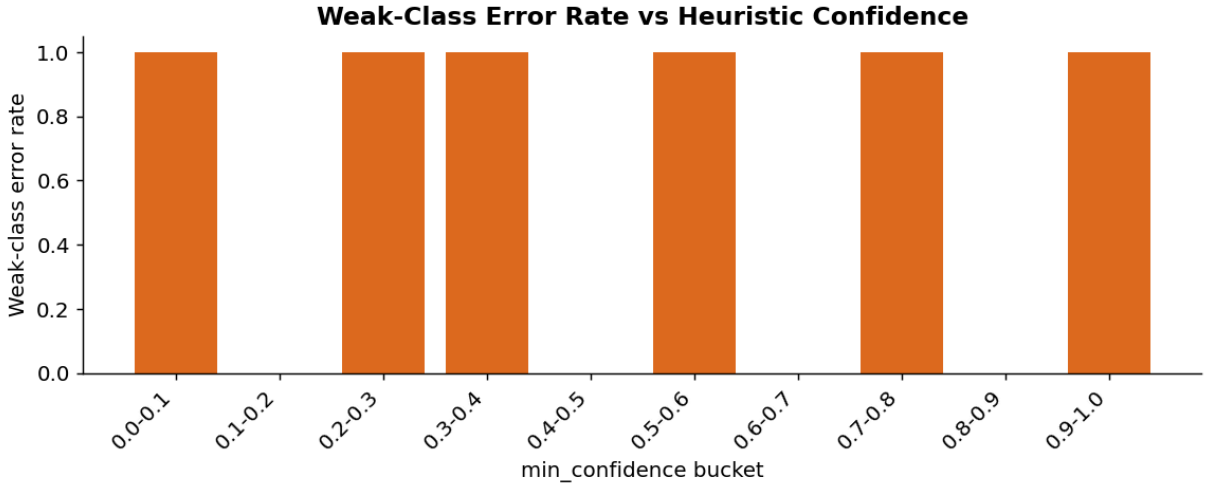
ml_split	joint_label	train	val	test
0	fmap::epi	27	6	5
1	fmap::fmapOtherDerived	18	1	3
2	fmap::fmapSusceptibilityOrPhaseMap	5	2	0
3	func::bold	4	1	2
4	func::funcStatMap	3	0	1
5	perf::perfCbvLikeMap	8	0	2
6	perf::perfTimingMap	17	5	3



In [24]:
```python
# Weak-class error rate vs heuristic min_confidence bucket
bucket_df = weak_eval.copy()
bucket_df = bucket_df[bucket_df["min_confidence"].notna()].copy()

if len(bucket_df) == 0:
    print("No min_confidence values available for weak-class bucket analysis
else:
    bins = np.linspace(0, 1, 11)
    labels = [f"{bins[i]:.1f}–{bins[i+1]:.1f}" for i in range(len(bins) - 1)
    bucket_df["conf_bucket"] = pd.cut(
        bucket_df["min_confidence"], bins=bins, labels=labels, include_lowes
    )
    bucket_stats = (
        bucket_df.groupby("conf_bucket", observed=False)["is_correct"]
```

```
        .agg(rows="count", accuracy="mean")
        .reset_index()
    )
    bucket_stats["error_rate"] = 1 - bucket_stats["accuracy"]

    display(bucket_stats)

    fig, ax = plt.subplots(figsize=(9, 3.8))
    ax.bar(bucket_stats["conf_bucket"].astype(str), bucket_stats["error_rate
    ax.set_xlabel("min_confidence bucket")
    ax.set_ylabel("Weak-class error rate")
    ax.set_ylim(0, 1.05)
    ax.set_title("Weak-Class Error Rate vs Heuristic Confidence", fontweight
    plt.xticks(rotation=45, ha="right")
    plt.tight_layout()
    plt.show()
    plt.close(fig)
```

	conf_bucket	rows	accuracy	error_rate
0	0.0-0.1	3	0.0000	1.0000
1	0.1-0.2	0	NaN	NaN
2	0.2-0.3	3	0.0000	1.0000
3	0.3-0.4	2	0.0000	1.0000
4	0.4-0.5	0	NaN	NaN
5	0.5-0.6	4	0.0000	1.0000
6	0.6-0.7	0	NaN	NaN
7	0.7-0.8	3	0.0000	1.0000
8	0.8-0.9	0	NaN	NaN
9	0.9-1.0	1	0.0000	1.0000



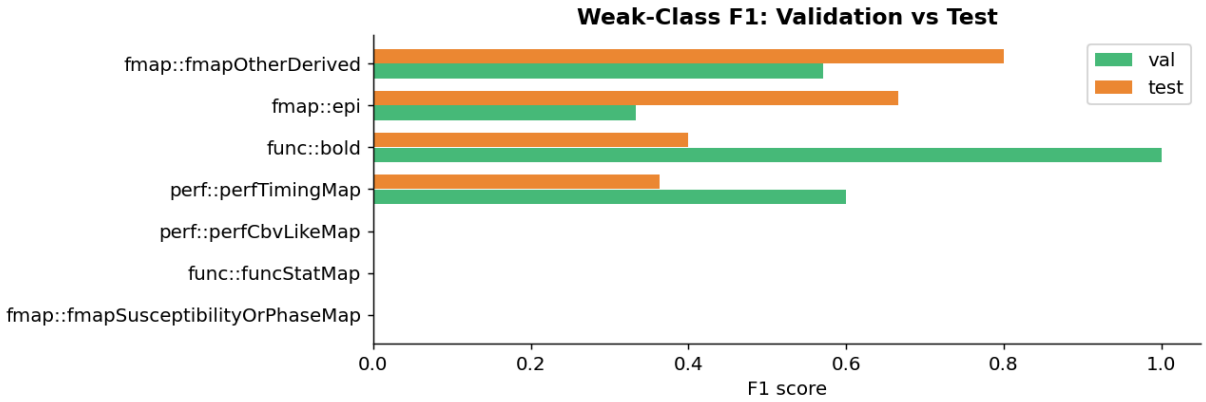
Part 8C — Stability Across Validation vs Test

This view separates persistent weak classes from split-instability artifacts by comparing validation and test F1 with explicit support counts.

In [25]:
```
# Validation vs test stability for weak classes
val_cls = report_val[~report_val["label"].isin(_summary_labels)].copy()
val_cls["support"] = val_cls["support"].astype(int)

test_cls = class_report[["label", "f1-score", "support"]].rename(
    columns={"f1-score": "f1_test", "support": "support_test"}
)
val_cls = val_cls[["label", "f1-score", "support"]].rename(
    columns={"f1-score": "f1_val", "support": "support_val"}
)

stability = test_cls.merge(val_cls, on="label", how="outer")
stability = stability[stability["label"].isin(weak_labels)].copy()
stability["f1_delta_test_minus_val"] = stability["f1_test"] - stability["f1_

# Tag interpretation buckets
stability["stability_tag"] = "stable_weak"
stability.loc[
    (stability["support_test"].fillna(0) < 5) | (stability["support_val"].fi
    "stability_tag",
] = "sparse_support"
stability.loc[
    (stability["f1_delta_test_minus_val"].abs() > 0.25)
    & ~((stability["support_test"].fillna(0) < 5) | (stability["support_val"
    "stability_tag",
] = "split_instability"

stability = stability.sort_values("f1_test", ascending=True)
print("Weak-class val/test stability:")
display(stability)

fig, ax = plt.subplots(figsize=(10, max(3.5, 0.5 * len(stability))))
y = np.arange(len(stability))
ax.barh(y - 0.18, stability["f1_val"].fillna(0), height=0.35, label="val", c
ax.barh(y + 0.18, stability["f1_test"].fillna(0), height=0.35, label="test",
ax.set_yticks(y)
ax.set_yticklabels(stability["label"])
ax.set_xlim(0, 1.05)
ax.set_xlabel("F1 score")
ax.set_title("Weak-Class F1: Validation vs Test", fontweight="bold")
ax.legend()
plt.tight_layout()
plt.show()
plt.close(fig)
```

```
Weak-class val/test stability:
```

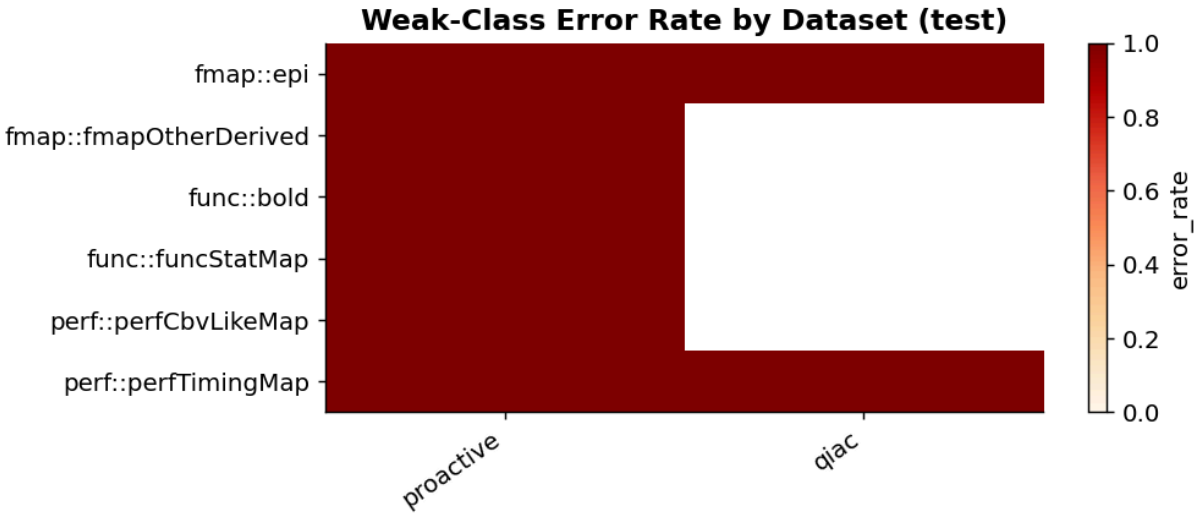
	label	f1_test	support_test	f1_val	support_val	f1_d
14	fmap::fmapSusceptibilityOrPhaseMap	0.0000	1.0000	0.0000	2.0000	
18	func::funcStatMap	0.0000	3.0000	NaN	NaN	
23	perf::perfCbvLikeMap	0.0000	3.0000	NaN	NaN	
24	perf::perfTimingMap	0.3636	5.0000	0.6000	5.0000	
17	func::bold	0.4000	1.0000	1.0000	2.0000	
11	fmap::epi	0.6667	7.0000	0.3333	5.0000	
12	fmap::fmapOtherDerived	0.8000	3.0000	0.5714	2.0000	



In [26]:
```python
# Dataset/site effect for weak classes on test split
if "dataset" not in weak_eval.columns:
    print("No dataset column available for site-effect analysis.")
else:
    ds_err = (
        weak_eval.groupby(["dataset", true_col])["is_correct"]
        .agg(rows="count", accuracy="mean")
        .reset_index()
    )
    ds_err["error_rate"] = 1 - ds_err["accuracy"]
    ds_err = ds_err.sort_values(["dataset", "error_rate"], ascending=[True,

    print("Dataset-level weak-class error rates:")
    display(ds_err)

    pivot = ds_err.pivot(index=true_col, columns="dataset", values="error_ra
    fig, ax = plt.subplots(figsize=(8, max(3.5, 0.55 * len(pivot))))
    im = ax.imshow(pivot.fillna(np.nan).values, aspect="auto", cmap="OrRd",
    ax.set_yticks(np.arange(len(pivot.index)))
    ax.set_yticklabels(pivot.index)
    ax.set_xticks(np.arange(len(pivot.columns)))
    ax.set_xticklabels(pivot.columns, rotation=35, ha="right")
    ax.set_title("Weak-Class Error Rate by Dataset (test)", fontweight="bold
    cbar = fig.colorbar(im, ax=ax)
    cbar.set_label("error_rate")
    plt.tight_layout()
    plt.show()
    plt.close(fig)
```

Dataset-level weak-class error rates:

	dataset	joint_label	rows	accuracy	error_rate
0	proactive	fmap::epi	4	0.0000	1.0000
1	proactive	fmap::fmapOtherDerived	3	0.0000	1.0000
2	proactive	func::bold	2	0.0000	1.0000
3	proactive	func::funcStatMap	1	0.0000	1.0000
4	proactive	perf::perfCbvLikeMap	2	0.0000	1.0000
5	proactive	perf::perfTimingMap	2	0.0000	1.0000
6	qiac	fmap::epi	1	0.0000	1.0000
7	qiac	perf::perfTimingMap	1	0.0000	1.0000

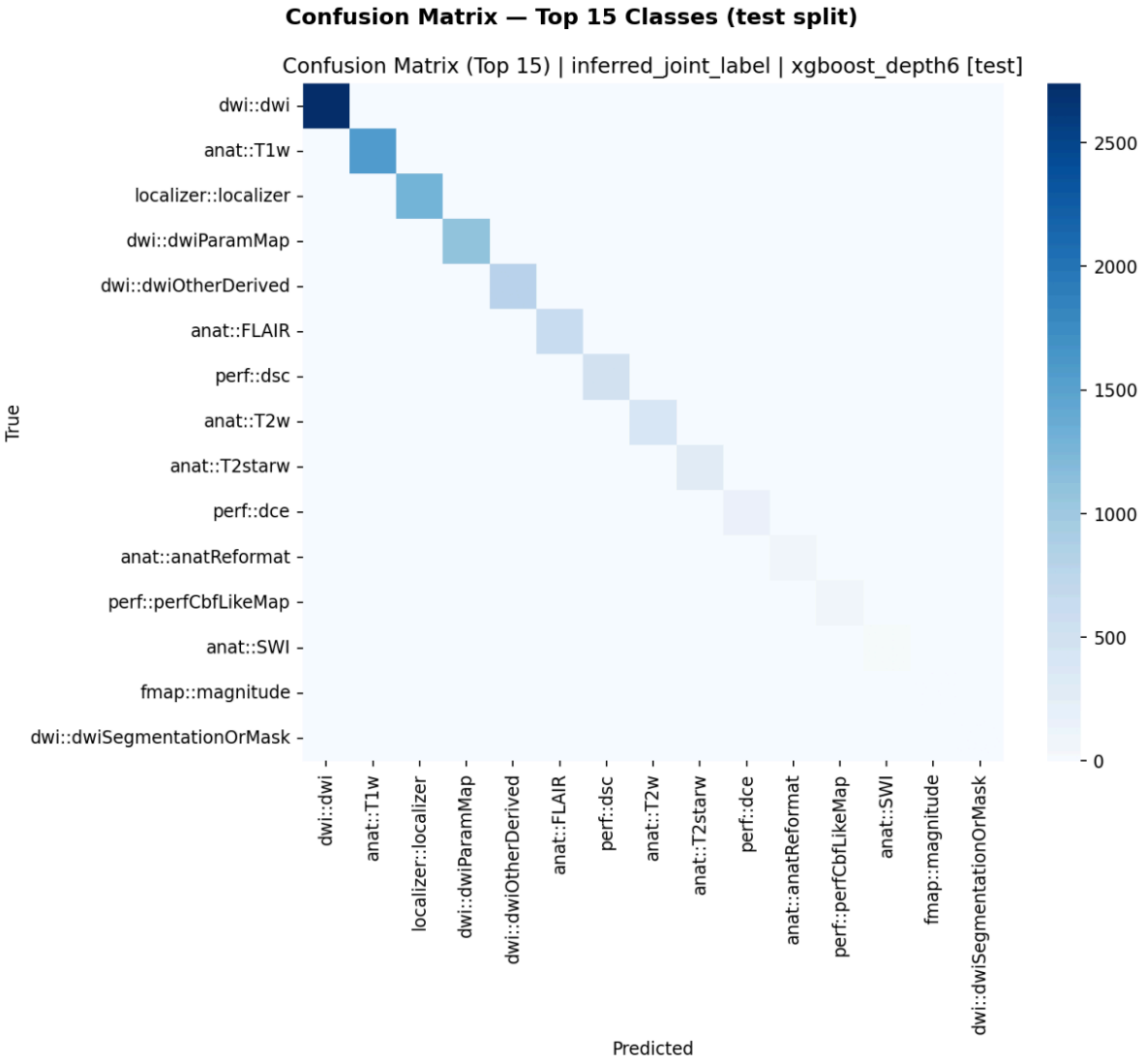


Part 9 — Confusion Analysis

The confusion matrix below shows the **top 15 most frequent classes** to keep it readable. Off-diagonal cells represent misclassifications; most errors stay within the same datatype family (e.g., `anat::T1w` being confused with `anat::T2w`), which suggests the model understands broad scan families well but occasionally struggles to distinguish closely related subtypes.

In [16]:
```
confusion_img_path = CONFUSION_DIR / "confusion_top15_inferred_joint_label_t
if confusion_img_path.exists():
    img = mpimg.imread(str(confusion_img_path))
    fig, ax = plt.subplots(figsize=(11, 9))
    ax.imshow(img)
    ax.axis("off")
    ax.set_title("Confusion Matrix — Top 15 Classes (test split)", fontweigh
    plt.tight_layout()
    plt.show()
```

```
    plt.close(fig)
else:
    print(f"Image not found: {confusion_img_path}")
    print("Re-run ml.ipynb cell #VSC-c5d18b69 to generate it.")
```



Part 10 — Unknown-Label Inference

Series that the heuristic engine could not label confidently (either `inferred_datatype` or `inferred_suffix` is `unknown`) were **excluded from supervised training** but were **scored by the final model** in a separate inference pass.

The review below shows what the model predicts for these heuristically ambiguous series and how confident those predictions are. High-confidence ML predictions on unknown-heuristic rows are strong candidates for accepting as BIDS labels — or for routing to human review with a suggested label.

In [34]:
```
print(f"Unknown-heuristic series scored by ML: {len(unknown_review):,}")
display(unknown_review[[
    "dataset", "subject_id",
    # "series_id",
    "series_description",
    "inferred_datatype", "inferred_suffix", "min_confidence",
    "pred_inferred_joint_label", "pred_inferred_joint_label_confidence",
]].head(15))
```

```
Unknown-heuristic series scored by ML: 12,527
```

	dataset	subject_id	series_description	inferred_datatype	inferred_suffix	min
0	proactive	1022-1937	Ax T1	anat	unknown	
1	proactive	1022-1937	Ax DWI_TRACEW	dwi	unknown	
2	proactive	1022-1937	Ax DTI_TRACEW	dwi	unknown	
3	proactive	1022-1937	Trace:Sep 12 2023 12-09-32 CDT	dwi	unknown	
4	proactive	1022-1937	AvDC:Sep 12 2023 12-09-33 CDT	dwi	unknown	
5	proactive	1022-1937	Universal:352:AvDC:Sep 12 2023 12-09-33 CDT	dwi	unknown	
6	proactive	1022-1937	Trace:Jul 14 2023 16-46-39 CDT	dwi	unknown	
7	proactive	1022-1937	AvDC:Jul 14 2023 16-46-41 CDT	dwi	unknown	
8	proactive	1022-1937	Universal:552:AvDC:Jul 14 2023 16-46-41 CDT	dwi	unknown	
9	proactive	1022-1937	COL:MRA Head	dwi	unknown	
10	proactive	1022-1937	Axial MIP 10_1	dwi	unknown	
11	proactive	1022-1937	Sag MIP 10_1	dwi	unknown	
12	proactive	1022-1937	Cor MIP 10_1	dwi	unknown	
13	proactive	1022-1937	AvDC:Jul 18 2023 20-20-16 CDT	dwi	unknown	
14	proactive	1022-1937	Universal:352:AvDC:Jul 18 2023 20-20-16 CDT	dwi	unknown	

Weak-Class Opportunity In Unknown-Heuristic Series

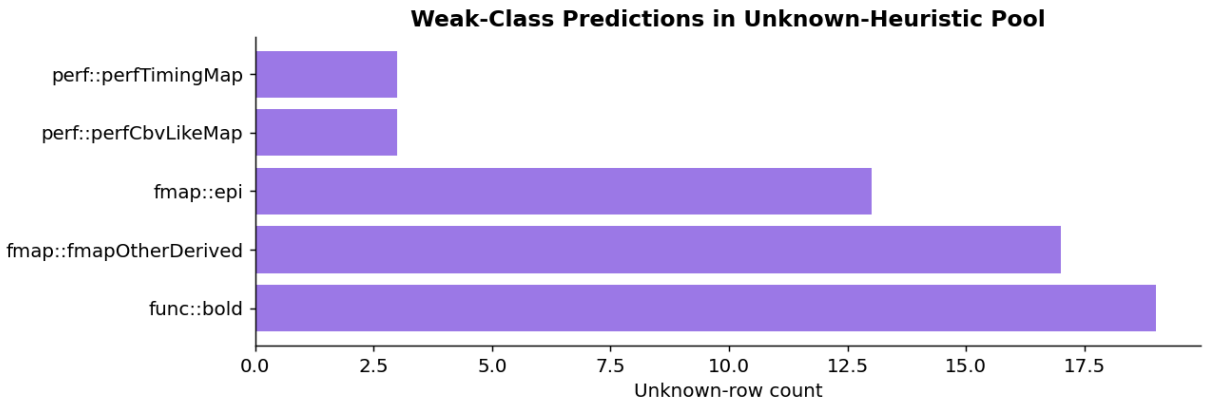
How often does the model predict weak classes on unknown-heuristic rows, and at what confidence? This identifies candidates for targeted human review and class expansion.

In [27]:
```
# Unknown-series opportunities for weak classes
unknown_pred_col = "pred_inferred_joint_label"
unknown_conf_col = "pred_inferred_joint_label_confidence"
```

```
weak_unknown = unknown_review[
    unknown_review[unknown_pred_col].isin(weak_labels)
].copy()

if len(weak_unknown) == 0:
    print("No unknown-heuristic rows predicted as weak classes.")
else:
    weak_unknown_summary = (
        weak_unknown.groupby(unknown_pred_col)[unknown_conf_col]
        .agg(
            rows="count",
            mean_conf="mean",
            median_conf="median",
            ge_090=lambda s: (s >= 0.90).sum(),
            ge_095=lambda s: (s >= 0.95).sum(),
        )
        .reset_index()
        .sort_values("rows", ascending=False)
    )
    weak_unknown_summary["pct_ge_090"] = weak_unknown_summary["ge_090"] / we
    weak_unknown_summary["pct_ge_095"] = weak_unknown_summary["ge_095"] / we

    print(f"Unknown rows predicted as weak classes: {len(weak_unknown):,}")
    display(weak_unknown_summary)

    fig, ax = plt.subplots(figsize=(10, max(3.5, 0.5 * len(weak_unknown_summ
    ax.barh(weak_unknown_summary[unknown_pred_col], weak_unknown_summary["rc
    ax.set_xlabel("Unknown-row count")
    ax.set_title("Weak-Class Predictions in Unknown-Heuristic Pool", fontwei
    plt.tight_layout()
    plt.show()
    plt.close(fig)
```

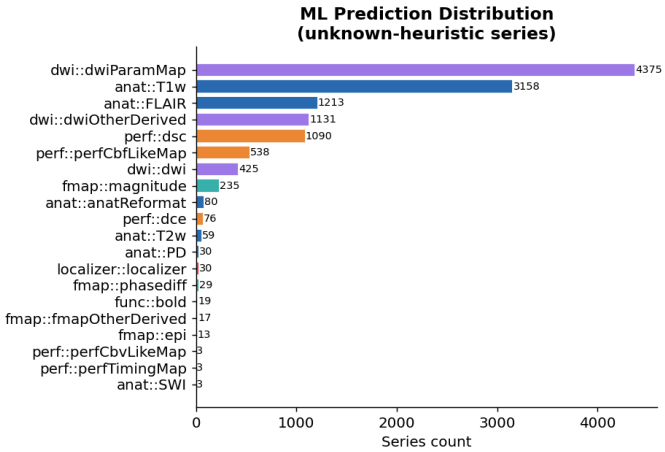
```
Unknown rows predicted as weak classes: 55
```

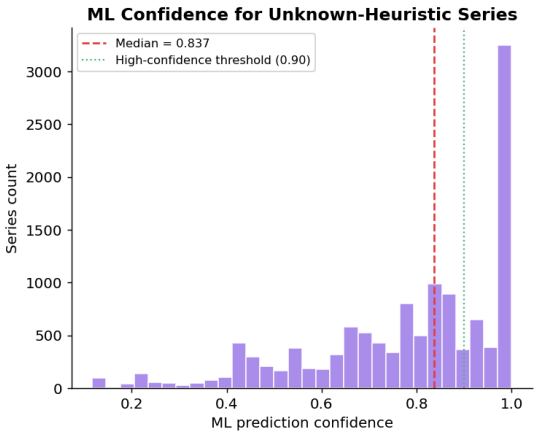
	pred_inferred_joint_label	rows	mean_conf	median_conf	ge_090	ge_095	pct_ge_
2	func::bold	19	0.1977	0.1930	0	0	0.0
1	fmap::fmapOtherDerived	17	0.4080	0.3653	0	0	0.0
0	fmap::epi	13	0.3128	0.2795	0	0	0.0
3	perf::perfCbvLikeMap	3	0.6676	0.7287	0	0	0.0
4	perf::perfTimingMap	3	0.3849	0.3564	0	0	0.0



In [18]:

```
pred_col  = "pred_inferred_joint_label"
conf_col  = "pred_inferred_joint_label_confidence"

pred_counts = unknown_review[pred_col].value_counts().head(20).sort_values()
pred_dtypes = pred_counts.index.str.split("::").str[0]
pred_colors = [_family_palette.get(d, "#A0AEC0") for d in pred_dtypes]

fig, axes = plt.subplots(1, 2, figsize=(13, 5))

ax = axes[0]
ax.barh(pred_counts.index, pred_counts.values, color=pred_colors, edgecolor=
for i, v in enumerate(pred_counts.values):
    ax.text(v + 0.3, i, str(v), va="center", fontsize=8)
ax.set_xlabel("Series count")
ax.set_title("ML Prediction Distribution\n(unknown-heuristic series)", fontw

ax2 = axes[1]
conf_vals = unknown_review[conf_col].dropna()
ax2.hist(conf_vals, bins=30, color="#9F7AEA", edgecolor="white", alpha=0.85)
ax2.axvline(conf_vals.median(), color="#E53E3E", linewidth=1.5, linestyle="-
            label=f"Median = {conf_vals.median():.3f}")
ax2.axvline(0.90, color="#48BB78", linewidth=1.2, linestyle=":",
            label="High-confidence threshold (0.90)")
ax2.set_xlabel("ML prediction confidence")
ax2.set_ylabel("Series count")
ax2.set_title("ML Confidence for Unknown-Heuristic Series", fontweight="bold
ax2.legend(fontsize=9)

plt.tight_layout()
plt.show()
plt.close(fig)

high_conf_n = int((conf_vals >= 0.90).sum())
print(f"ML confidence ≥ 0.90: {high_conf_n:,} / {len(conf_vals):,}  ({high_c
print(f"Median confidence   : {conf_vals.median():.4f}")
print(f"Mean confidence     : {conf_vals.mean():.4f}")
```






```
ML confidence ≥ 0.90: 4,460 / 12,527  (35.6%)
Median confidence   : 0.8371
Mean confidence     : 0.7805
```

In [19]:
```
# High-confidence ML predictions — candidates for auto-accepting as BIDS lab
high_conf_rows = unknown_review[unknown_review[conf_col] >= 0.90].sort_value
print(f"High-confidence ML predictions (confidence ≥ 0.90): {len(high_conf_r
display(high_conf_rows[[
    "dataset", "series_id",
    "inferred_datatype", "inferred_suffix",
    "series_description",
    pred_col, conf_col,
]].head(20))
```

```
High-confidence ML predictions (confidence ≥ 0.90): 4,460 rows
```

	dataset	series_id	inferred_datatype	i
12232	qiac	1.3.12.2.1107.5.2.50.176522.300000250806172503...	anat	
4466	proactive	1.2.840.113619.2.408.5.14196467.865465.19502.1...	dwi	
6226	qiac	1.3.12.2.1107.5.2.18.142582.300000250601095642...	anat	
5863	proactive	1.3.12.2.1107.5.2.50.176519.202309292201159345...	unknown	
1292	proactive	1.3.12.2.1107.5.2.18.52113.2023010508273779931...	dwi	
8839	qiac	1.2.840.113619.2.80.0.23910.1745459832.63.13.2	dwi	
2724	proactive	1.2.840.113619.2.80.3037550886.1207.1662679678...	dwi	
781	proactive	1.2.840.113619.2.80.2077619178.1179.1639089091...	dwi	
12317	qiac	1.2.840.113619.2.80.0.15015.1746049600.63.13.2	dwi	
2717	proactive	1.2.840.113619.2.80.3037550886.30935.166267905...	dwi	
3449	proactive	2.25.114510977345967483199948353585188022550	fmap	
11047	qiac	1.3.12.2.1107.5.2.18.141126.300000250710181617...	unknown	
4188	proactive	1.3.12.2.1107.5.2.18.52115.2023081007050054552...	dwi	
2571	proactive	1.2.840.113619.2.80.0.54150.1675882666.69.13.2	dwi	
12485	qiac	1.3.12.2.1107.5.2.18.52115.3000002507171101377...	anat	
4830	proactive	1.3.12.2.1107.5.2.18.52107.2024012207433252672...	anat	
12481	qiac	1.2.840.113619.2.80.0.12498.1757010811.105.13.2	dwi	
12399	qiac	1.3.12.2.1107.5.2.18.152222.300000250604124847...	unknown	
8823	qiac	1.3.12.2.1107.5.2.18.142098.300000250731125113...	unknown	
1005	proactive	1.3.6.1.4.1.34261.3.181124907717723.15.27400.2...	dwi	

Part 11 — Localizer Normalization

A dedicated pre-training normalization step converts `localizer::unknown` (series the heuristic identifies as a localizer but cannot assign a specific suffix) to the canonical `localizer::localizer` label.

Without this step, the most common localizer subtype would be silently dropped by the `drop_unknown_in_train_val` filter, starving the model of localizer training signal.

In [20]:
```
localizer_rows = analysis_df[
    analysis_df["inferred_datatype"].fillna("").str.lower().eq("localizer")
]
vc_loc = localizer_rows["joint_label"].value_counts()
loc_dist = pd.DataFrame({"joint_label": vc_loc.index, "count": vc_loc.values
print(f"Total localizer-family rows: {len(localizer_rows):,}")
display(loc_dist)

if "localizer::localizer" in class_report["label"].values:
    loc_row = class_report[class_report["label"] == "localizer::localizer"].
    print(
        f"\nlocalizer::localizer test — "
        f"precision={loc_row['precision']:.3f}, recall={loc_row['recall']:.3
        f"F1={loc_row['f1-score']:.3f}, support={int(loc_row['support'])}"
    )
```

```
Total localizer-family rows: 16,340
```

	joint_label	count
0	localizer::localizer	16340

```
localizer::localizer test — precision=1.000, recall=0.996, F1=0.998, support
=1309
```

Key Takeaways

Strengths

- **99.5% test accuracy / 99.9% top-2 accuracy** — the correct label is almost always the top or second prediction.
- **0.999 macro AUC** — near-perfect discrimination across virtually all classes.
- Strong performance on the most clinically important classes: `anat::T1w`, `anat::T2w`, `dwi::dwi`, `func::bold`, `localizer::localizer`.
- Confidence weighting improves minority-class recall without sacrificing majority-class precision.

Limitations

- **78.4% macro F1** — lower than weighted accuracy because several rare classes have very few test samples (1–5 each).
- Six classes have F1 = 0 in the test fold, driven entirely by data sparsity, not model failure.
- Unknown-series ML inference is promising but not yet validated against ground-truth BIDS labels — human review of high-confidence predictions is recommended

before auto-accepting them.

Recommended Next Steps

1. Collect more labeled examples for rare classes (`func::funcStatMap` , `perf::perfCbvLikeMap` , `perf::perfTimingMap` , `fmap::fmapSusceptibilityOrPhaseMap`)
2. Run human review on unknown-label series with ML confidence ≥ 0.90 to expand the labeled dataset
3. Consider adding prediction calibration (Platt scaling or isotonic regression) to improve the reliability of confidence scores for downstream decision thresholds